PRAKTIKUM MIN PROJECT

Dosen Pengampu: Isbat Uzzin Nadhori S.Kom., M.T



Disusun untuk Memenuhi Tugas

Mata Kuliah:

PRAKTIKUM DATA WAREHOUSE

Oleh:

Dea Aulia Rifda Yuniar

3324600047

3 Sains Data Terapan B

DEPARTEMEN TEKNIK INFORMATIKA DAN KOMPUTER

POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

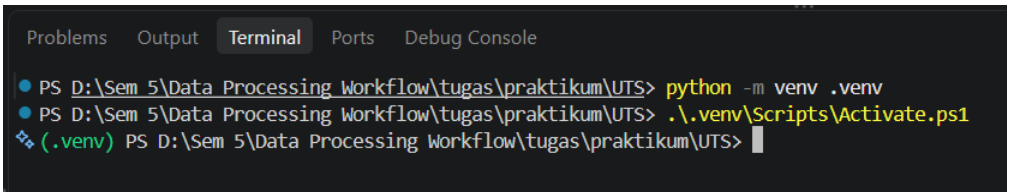
OKTOBER 2026

A. MEMPERSIAPKAN LINGKUNGAN

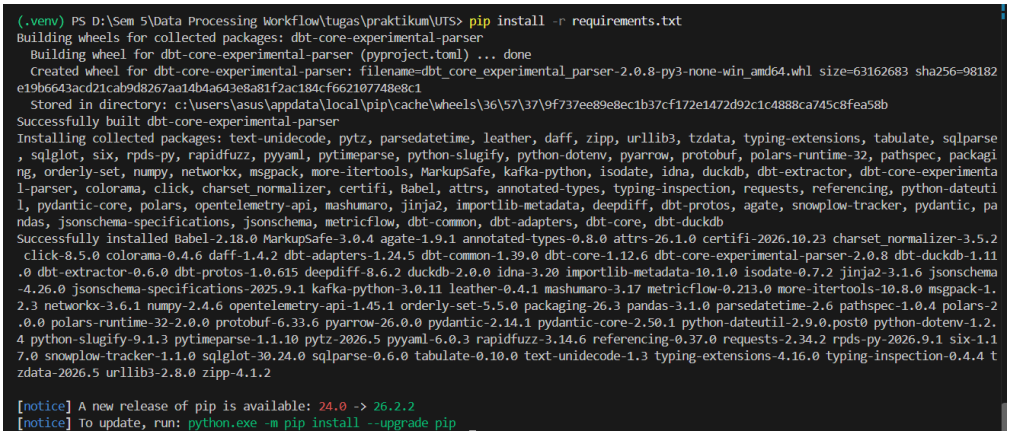
1. Buat virtual environment python

FOLDER :

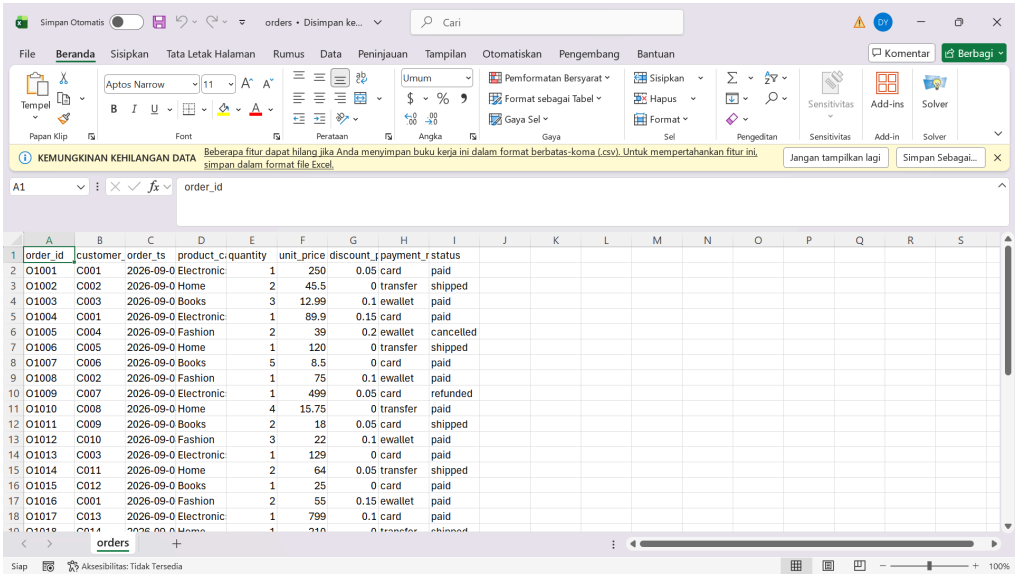




INSTALASI LIBRARY



2. Periksa dataset

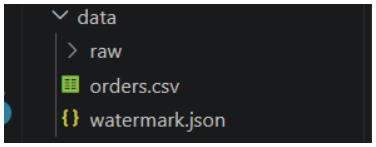


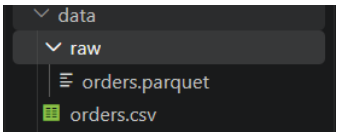
MENGECEK KETERSEDIAAN DOCKER

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> docker --version
Docker version 28.1.1, build 4eba377
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> docker compose version
```

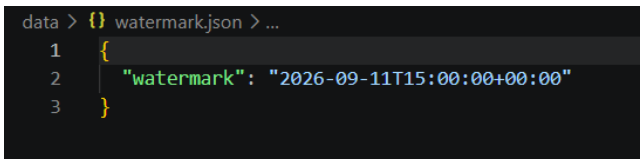
3. Jalankan ekstraksi batch simulasi API

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> python scripts/batch_ingest.py --source data/orders.csv --out data/raw
page=0 rows=25
page=1 rows=11
output=data\raw\orders.parquet total_unique_rows=36
Note: this educational adapter paginates a local CSV; replace get_page with a real requests adapter for an actual REST API.
```



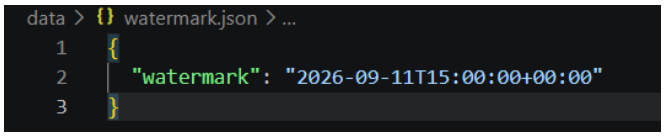


ISI watermark.json



MENGUJI “Apakah script dapat dijalankan dua kali tanpa menghasilkan duplikat data” DENGAN MENJALANKAN KEMBALI SCRIPT INGESTION

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> python scripts/batch_ingest.py --source data/orders.csv --out data/raw
page=0 rows=25
page=1 rows=11
output=data\raw\orders.parquet total_unique_rows=36
Note: this educational adapter paginates a local CSV; replace get_page with a real requests adapter for an actual REST API.
```



Hasilnya pencegahan terhadap data duplikasi berhasil dilakukan. Hal ini dibuktikan dengan tidak ada penambahan jumlah row data(tetap 36 baris).

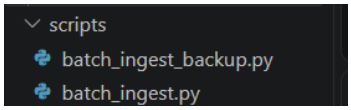
TEMUAN :

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> python -c "import pandas as pd; df=pd.read_csv('data/orders.csv'); df['order_ts']=pd.to_datetime(df['order_ts'], errors='coerce', utc=True); print('Jumlah baris:', len(df)); print('Timestamp terbesar:', df['order_ts'].max()); print('Jumlah timestamp kosong:', df['order_ts'].isna().sum())"
Jumlah baris: 36
Timestamp terbesar: 2026-09-11 15:00:00+00:00
Jumlah timestamp kosong: 1
```

Artinya ada satu data yang watermark atau timestampnya tidak terbaca. Hal ini dapat terjadi karena data itu tidak memiliki timestamp atau timestampnya tidak sesuai ketentuan. Sehingga saat data ini dimasukkan sebagai data raw timestamp tersebut akan diisi/diubah menjadi berikut :

```
Jumlah timestamp kosong: 1
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> python -c "import pandas as pd; df=pd.read_csv('data/orders.csv'); df['order_ts_parsed']=pd.to_datetime(df['order_ts'], errors='coerce', utc=True); print(df.loc[df['order_ts_parsed'].isna(), ['order_id','customer_id','order_ts']].to_string(index=False))"
order_id customer_id   order_ts
   O1034        C029 not-a-date
```

MELAKUKAN BACK-UP FILE batch_ingest.py untuk uji retry




```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> Copy-Item scripts/batch_ingest.py scripts/batch_ingest_backup.py
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS>
```

UBAH isi batch_ingest.py seperti ini

scripts > batch_ingest.py > ...

```python
"""Batch ingestion: paginated CSV source simulating a REST API."""
import argparse
import json
import time
from pathlib import Path

import pandas as pd


def get_page(
    df,
    page,
    page_size,
    retries=3,
    fail_first_attempts=0,
):
    """Read one page and optionally simulate temporary failures."""
    start = page * page_size

    for attempt in range(retries):
        try:
            # Testing aid: simulate temporary errors before success.
            if attempt < fail_first_attempts:
                raise RuntimeError("Simulated temporary API failure")

            return df.iloc[start:start + page_size].copy()

        except Exception as exc:
            if attempt == retries - 1:
                raise
            wait_seconds = 2 ** attempt
            print(
                f"page={page} attempt={attempt + 1} "
                f"error={exc}; retry in {wait_seconds}s"
            )
            time.sleep(wait_seconds)


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--source", default="data/orders.csv")
    parser.add_argument("--out", default="data/raw")
    parser.add_argument("--page-size", type=int, default=25)
    parser.add_argument("--checkpoint", default="data/watermark.json")
    parser.add_argument(
        "--simulate-failures",
        type=int,
        default=0,
        help="Number of temporary failures simulated per page",
    )
    args = parser.parse_args()

    if args.page_size <= 0:
        parser.error("--page-size must be greater than zero")

    if args.simulate_failures < 0:
        parser.error("--simulate-failures cannot be negative")

    if args.simulate_failures >= 3:
        parser.error(
            "--simulate-failures must be less than 3 "
            "because the retry limit is 3 attempts"
        )
```

```
    source = pd.read_csv(args.source)

    if "order_id" not in source.columns:
        raise ValueError("Source must contain order_id")

    if "order_ts" in source.columns:
        source["order_ts"] = pd.to_datetime(
            source["order_ts"], errors="coerce", utc=True
        )

    out_dir = Path(args.out)
    out_dir.mkdir(parents=True, exist_ok=True)

    checkpoint = Path(args.checkpoint)
    target = out_dir / "orders.parquet"

    # Read all source pages; the CSV acts as a paginated API.
    pages = []
    page = 0
    rows_read = 0

    while True:
        chunk = get_page(
            source,
            page,
            args.page_size,
            retries=3,
            fail_first_attempts=args.simulate_failures,
        )

        if chunk.empty:
            break
```

```
        rows_read += len(chunk)
        print(f"page={page} rows={len(chunk)}")
        pages.append(chunk)
        page += 1

    incoming = (
        pd.concat(pages, ignore_index=True)
        if pages
        else source.iloc[0:0].copy()
    )

    # Keep the latest occurrence of each order_id in the incoming batch.
    incoming = incoming.drop_duplicates("order_id", keep="last")

    # Load existing raw data, if present.
    if target.exists():
        existing = pd.read_parquet(target)
    else:
        existing = incoming.iloc[0:0].copy()

    existing_ids = set(existing["order_id"].dropna().astype(str))
    incoming_ids = incoming["order_id"].astype("string")

    # Use order_id for sink deduplication so equal or late timestamps
    # do not cause unseen orders to be skipped.
    new_rows = incoming.loc[
        incoming_ids.notna() & ~incoming_ids.isin(existing_ids)
    ].copy()

    combined = pd.concat([existing, new_rows], ignore_index=True)
    combined = combined.drop_duplicates("order_id", keep="last")

    combined.to_parquet(target, index=False)
```

```python
    # It is not used alone to decide whether an order is new.
    watermark = None
    if "order_ts" in combined.columns:
        valid_ts = pd.to_datetime(
            combined["order_ts"], errors="coerce", utc=True
        ).dropna()

        if not valid_ts.empty:
            watermark = valid_ts.max().isoformat()

    checkpoint.parent.mkdir(parents=True, exist_ok=True)
    checkpoint.write_text(
        json.dumps({"watermark": watermark}, indent=2),
        encoding="utf-8",
    )

    # Full contract validation/rejection is performed in the next stage.
    rejected = 0
    successful = rows_read
    added = len(new_rows)

    print("\n=== INGESTION SUMMARY ===")
    print(f"rows_read={rows_read}")
    print(f"rows_successful={successful}")
    print(f"rows_rejected_at_ingestion={rejected}")
    print(f"rows_added={added}")
    print(f"total_unique_rows={len(combined)}")
    print(f"output={target}")
    print(f"watermark={watermark}")
    print(
        "Note: this is a local CSV pagination simulation, "
        "not a live REST API."
    )


if __name__ == "__main__":
    main()
```

UJI COBA INGESTION NORMAL

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> python scripts/batch_ingest.py --source data/orders.csv --out data/raw
page=0 rows=25
page=1 rows=11

=== INGESTION SUMMARY ===
rows_read=36
rows_successful=36
rows_rejected_at_ingestion=0
rows_added=0
total_unique_rows=36
output=data\raw\orders.parquet
watermark=2026-09-11T15:00:00+00:00
Note: this is a local CSV pagination simulation, not a live REST API.
```

UJI RETRY

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> python scripts/batch_ingest.py --source data/orders.csv --out data/raw --simu
late-failures 1
page=0 attempt=1 error=Simulated temporary API failure; retry in 1s
page=0 rows=25
page=1 attempt=1 error=Simulated temporary API failure; retry in 1s
page=1 rows=11
page=2 attempt=1 error=Simulated temporary API failure; retry in 1s

=== INGESTION SUMMARY ===
rows_read=36
rows_successful=36
rows_rejected_at_ingestion=0
rows_added=0
total_unique_rows=36
output=data\raw\orders.parquet
watermark=2026-09-11T15:00:00+00:00
Note: this is a local CSV pagination simulation, not a live REST API.
```

proses akan membutuhkan beberapa detik lebih lama. Setelah retry berhasil, jumlah data tetap 36 dan tidak ada order baru yang ditambahkan.

BUKTI BAHWA DATA TIDAK TERDUPLIKASI

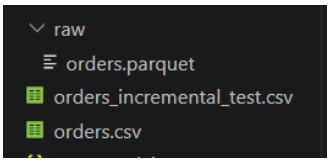
```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> python -c "import pandas as pd; df=pd.read_parquet('data/raw/orders.parquet')
; print('Total baris:',len(df)); print('ID unik:',df['order_id'].nunique()); print('ID duplikat:',df['order_id'].duplicated().sum())"
Total baris: 36
ID unik: 36
ID duplikat: 0
```

Kesesuaian antara jumlah baris dan total data unik menandakan tidak adanya data duplikasi.

TAMBAHAN PENGUJIAN BAHWA WATERMARK HANYA AKAN MENYIMPAN DATA BARU (TIMESTAMP TERBESAR)

a. Buat sumber pengujian

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> python -c "import pandas as pd; df=pd.read_csv('data/orders.csv'); new=df.iloc[[-1]].copy(); new.loc[new.index[0],'order_id']='TEST_INCREMENTAL_001'; new.loc[new.index[0],'order_ts']='2026-09-12T00:00:00Z'; df=pd.concat([df,new],ignore_index=True); df.to_csv('data/orders_incremental_test.csv',index=False); print('Baris sumber pengujian:',len(df)); print('Order uji:',new.iloc[0]['order_id']); print('Timestamp uji:',new.iloc[0]['order_ts'])"
Baris sumber pengujian: 37
Order uji: TEST_INCREMENTAL_001
Timestamp uji: 2026-09-12T00:00:00Z
```



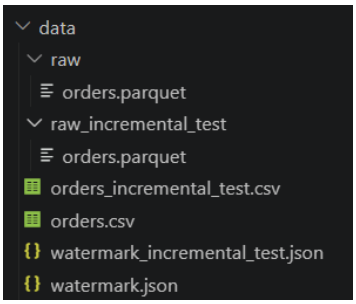
b. Siapkan salinan Parquet untuk pengujian dan watermark.json

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> New-Item -ItemType Directory -Force data/raw_incremental_test


    Directory: D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\data


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        10/25/2026  12:33 AM                raw_incremental_test

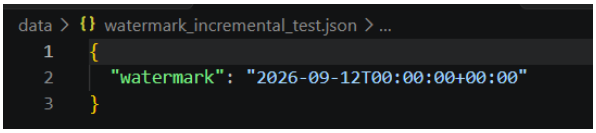

(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> Copy-Item data/raw/orders.parquet data/raw_incremental_test/orders.parquet
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> Copy-Item data/watermark.json data/watermark_incremental_test.json
```



c. Jalankan incremental ingestion

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> python scripts/batch_ingest.py --source data/orders_incremental_test.csv
 --out data/raw_incremental_test --checkpoint data/watermark_incremental_test.json
page=0 rows=25
page=1 rows=12

=== INGESTION SUMMARY ===
rows_read=37
rows_successful=37
rows_rejected_at_ingestion=0
rows_added=1
total_unique_rows=37
output=data\raw_incremental_test\orders.parquet
watermark=2026-09-12T00:00:00+00:00
Note: this is a local CSV pagination simulation, not a live REST API.
```



d. Verifikasi hasil

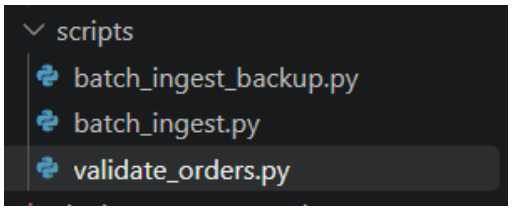
```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> python -c "import pandas as pd, json; df=pd.read_parquet('data/raw_incremental_test/orders.parquet'); print('Total baris:',len(df)); print('ID unik:',df['order_id'].nunique()); print('ID duplikat:',df['order_id'].duplicated().sum()); print('Order uji ditemukan:',(df['order_id']=='TEST_INCREMENTAL_001').sum()); print('Watermark:',json.load(open('data/watermark_incremental_test.json'))['watermark'])"
Total baris: 37
ID unik: 37
ID duplikat: 0
Order uji ditemukan: 1
Watermark: 2026-09-12T00:00:00+00:00
```

B. DATA CONTACT DAN FAIL-FAST

PERIKSA STRUKTUR RAW DATA

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> python -c "import pandas as pd; df=pd.read_parquet('data/raw/orders.parquet'); print('Kolom:',df.columns.tolist()); print('Jumlah record:',len(df)); print(df.dtypes); print(df.head(3).to_string(index=False))"
Kolom: ['order_id', 'customer_id', 'order_ts', 'product_category', 'quantity', 'unit_price', 'discount_pct', 'payment_method', 'status']
Jumlah record: 36
order_id                    str
customer_id                 str
order_ts          datetime64[us, UTC]
product_category            str
quantity                  int64
unit_price              float64
discount_pct            float64
payment_method              str
status                      str
dtype: object
order_id customer_id                order_ts product_category  quantity  unit_price  discount_pct payment_method  status
   O1001        C001 2026-09-01 09:15:00+00:00      Electronics         1      250.00          0.05           card    paid
   O1002        C002 2026-09-01 10:10:00+00:00            Home         2       45.50          0.00       transfer shipped
   O1003        C003 2026-09-01 11:05:00+00:00           Books         3       12.99          0.10        ewallet    paid
```

MEMBUAT FILE validate_orders.py



scripts > validate_orders.py > ...

```python

"""Validate raw orders using a Pydantic data contract."""
import json
from datetime import datetime
from pathlib import Path
from typing import Literal

import pandas as pd
from pydantic import BaseModel, ConfigDict, Field, ValidationError, field_validator


class OrderContract(BaseModel):
    model_config = ConfigDict(extra="allow")

    order_id: str
    customer_id: str
    order_ts: datetime
    quantity: int = Field(gt=0, strict=True)
    unit_price: float = Field(ge=0)
    discount_pct: float = Field(ge=0, le=1)
    status: Literal["paid", "shipped", "cancelled", "refunded"]

    @field_validator("order_id", "customer_id")
    @classmethod
    def require_nonempty_string(cls, value: str) -> str:
        value = value.strip()
        if not value:
            raise ValueError("Field must not be empty")
        return value

```

```python
def main():
    source = Path("data/raw/orders.parquet")
    valid_path = Path("data/processed/valid_orders.parquet")
    rejected_path = Path("data/rejected/rejected_orders.jsonl")
    summary_path = Path("data/processed/validation_summary.json")

    valid_path.parent.mkdir(parents=True, exist_ok=True)
    rejected_path.parent.mkdir(parents=True, exist_ok=True)

    df = pd.read_parquet(source)

    # Convert records to JSON-compatible values.
    # Invalid timestamps such as NaT become null and fail validation.
    records = json.loads(
        df.to_json(orient="records", date_format="iso")
    )

    valid_orders = []
    rejected_orders = []

    for record in records:
        try:
            order = OrderContract.model_validate(record)
            valid_orders.append(order.model_dump(mode="json"))

        except ValidationError as exc:
            errors = [
                {
                    "field": ".".join(str(part) for part in error["loc"]),
                    "message": error["msg"],
                    "type": error["type"],
                }
                for error in exc.errors()
            ]
```

```python
            rejected_orders.append(
                {
                    "order_id": record.get("order_id"),
                    "payload": record,
                    "errors": errors,
                }
            )

    # Save valid records in Parquet.
    valid_df = pd.DataFrame(valid_orders)
    if valid_df.empty:
        valid_df = pd.DataFrame(columns=df.columns)

    valid_df.to_parquet(valid_path, index=False)

    # Save rejected records as JSON Lines, including original payload
    # and actionable validation messages.
    with rejected_path.open("w", encoding="utf-8") as file:
        for rejected in rejected_orders:
            file.write(
                json.dumps(rejected, ensure_ascii=False) + "\n"
            )

    summary = {
        "source": str(source),
        "total_records": len(records),
        "valid_records": len(valid_orders),
        "rejected_records": len(rejected_orders),
        "warehouse_eligible_records": len(valid_orders),
        "valid_output": str(valid_path),
        "rejected_output": str(rejected_path),
        "validation_mode": "per-record quarantine",
    }
```

```python
    summary_path.write_text(
        json.dumps(summary, indent=2, ensure_ascii=False),
        encoding="utf-8",
    )

    print("\n=== DATA CONTRACT SUMMARY ===")
    print(f"total_records={len(records)}")
    print(f"valid_records={len(valid_orders)}")
    print(f"rejected_records={len(rejected_orders)}")
    print(f"valid_output={valid_path}")
    print(f"rejected_output={rejected_path}")
    print(f"summary_output={summary_path}")

    if rejected_orders:
        print("\n=== REJECTION DETAILS ===")
        for rejected in rejected_orders:
            print(f"order_id={rejected['order_id']}")
            for error in rejected["errors"]:
                print(
                    f"  field={error['field']}; "
                    f"reason={error['message']}"
                )


if __name__ == "__main__":
    main()
```

MEMBUAT TEMPAT UNTUK MENYIMPAN DATA YANG LOLOS VALIDASI DAN TIDAK LOLOS

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> New-Item -ItemType Directory -Force data/processed


    Directory: D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\data


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        10/25/2026  12:52 AM                processed

(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> New-Item -ItemType Directory -Force data/rejected

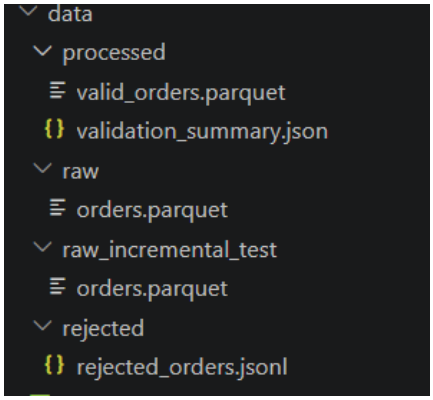

    Directory: D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\data


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        10/25/2026  12:52 AM                rejected
```

JALANKAN FILE validate_orders.py

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> python -u scripts/validate_orders.py
=== REJECTION DETAILS ===
order_id=O1031
  field=quantity; reason=Input should be greater than 0
order_id=O1032
  field=unit_price; reason=Input should be greater than or equal to 0
order_id=O1033
  field=discount_pct; reason=Input should be less than or equal to 1
order_id=O1035
  field=status; reason=Input should be 'paid', 'shipped', 'cancelled' or 'refunded'
order_id=O1036
  field=customer_id; reason=Input should be a valid string
order_id=O1034
  field=order_ts; reason=Input should be a valid datetime
```

PEMBUKTIAN :




```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> python -c "import json,pandas as pd; s=json.load(open('data/processed/validation_summary.json',encoding='utf-8')); print(json.dumps(s,indent=2)); print('Valid Parquet:',len(pd.read_parquet('data/processed/valid_orders.parquet'))); print('Jumlah baris rejected:',sum(1 for _ in open('data/rejected/rejected_orders.jsonl',encoding='utf-8')))"
{
  "source": "data\\raw\\orders.parquet",
  "total_records": 36,
  "valid_records": 30,
  "rejected_records": 6,
  "warehouse_eligible_records": 30,
  "valid_output": "data\\processed\\valid_orders.parquet",
  "rejected_output": "data\\rejected\\rejected_orders.jsonl",
  "validation_mode": "per-record quarantine"
}
Valid Parquet: 30
Jumlah baris rejected: 6
```

C. TRANSFORMASI DAN PARQUET

MEMBUAT FOLDER data/warehouse dan data/checpoints

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> New-Item -ItemType Directory -Force data/warehouse


    Directory: D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\data


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        10/25/2026   7:44 AM                warehouse


(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> New-Item -ItemType Directory -Force data/checkpoints


    Directory: D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\data


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        10/25/2026   7:45 AM                checkpoints
```

BUAT FILE TRANSFORMASINYA

scripts > transform_orders.py > ...

```python

"""Transform validated orders and store them as Parquet."""

import json
from pathlib import Path

import pandas as pd


def main():
    source = Path("data/processed/valid_orders.parquet")
    output = Path("data/processed/transformed_orders.parquet")
    summary_path = Path("data/processed/transformation_summary.json")

    if not source.exists():
        raise FileNotFoundError(
            f"Input tidak ditemukan: {source}. "
            "Jalankan validasi terlebih dahulu."
        )

    df = pd.read_parquet(source)

    # Standardisasi timestamp ke UTC.
    df["order_ts"] = pd.to_datetime(
        df["order_ts"],
        utc=True,
        errors="coerce",
    )

    if df["order_ts"].isna().any():
        invalid_count = int(df["order_ts"].isna().sum())
        raise ValueError(
            f"Ditemukan {invalid_count} timestamp yang tidak valid "
            "di dalam valid_orders.parquet."
        )

    # Pastikan kolom numerik menggunakan tipe yang konsisten.
    df["quantity"] = pd.to_numeric(df["quantity"], errors="raise").astype("int64")
    df["unit_price"] = pd.to_numeric(df["unit_price"], errors="raise").astype("float64")
    df["discount_pct"] = pd.to_numeric(df["discount_pct"], errors="raise").astype("float64")

    # Hitung metrik transaksi.
    df["gross_amount"] = df["quantity"] * df["unit_price"]
    df["net_amount"] = df["gross_amount"] * (1 - df["discount_pct"])

    # Simpan ke Parquet tanpa mengubah raw layer.
    output.parent.mkdir(parents=True, exist_ok=True)
    df.to_parquet(output, index=False)

    summary = {
        "source": str(source),
        "output": str(output),
        "record_count": len(df),
        "unique_order_ids": int(df["order_id"].nunique()),
        "duplicate_order_ids": int(df["order_id"].duplicated().sum()),
        "gross_amount_total": float(df["gross_amount"].sum()),
        "net_amount_total": float(df["net_amount"].sum()),
        "timestamp_timezone": "UTC",
        "numeric_types": {
            "quantity": str(df["quantity"].dtype),
            "unit_price": str(df["unit_price"].dtype),
            "discount_pct": str(df["discount_pct"].dtype),
            "gross_amount": str(df["gross_amount"].dtype),
            "net_amount": str(df["net_amount"].dtype),
        },
    }
```

```
    summary_path.write_text(
        json.dumps(summary, indent=2, ensure_ascii=False),
        encoding="utf-8",
    )

    print("=== TRANSFORMATION SUMMARY ===")
    print(json.dumps(summary, indent=2, ensure_ascii=False))
    print("\n=== SAMPLE RESULTS ===")
    print(
        df[
            [
                "order_id",
                "quantity",
                "unit_price",
                "discount_pct",
                "gross_amount",
                "net_amount",
                "order_ts",
            ]
        ].head().to_string(index=False)
    )


if __name__ == "__main__":
    main()
```

Script ini hanya membaca hasil validasi dan menulis hasil transformasi ke file baru. Dengan demikian, `data/raw/orders.parquet` tetap menjadi salinan data mentah yang dapat diaudit.

JALANKAN FILE TRANSFORMASI

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> python -m py_compile scripts/transform_orders.py
```

Code ini digunakan untuk memeriksa apakah ada kesalahan sintaks pada code, seperti kurangnya tanda kurung atau titik-koma.

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> python -u scripts/transform_orders.py
=== TRANSFORMATION SUMMARY ===
{
  "source": "data\\processed\\valid_orders.parquet",
  "output": "data\\processed\\transformed_orders.parquet",
  "record_count": 30,
  "unique_order_ids": 30,
  "duplicate_order_ids": 0,
  "gross_amount_total": 4245.83,
  "net_amount_total": 3948.473,
  "timestamp_timezone": "UTC",
  "numeric_types": {
    "quantity": "int64",
    "unit_price": "float64",
    "discount_pct": "float64",
    "gross_amount": "float64",
    "net_amount": "float64"
  }
}

=== SAMPLE RESULTS ===
order_id  quantity  unit_price  discount_pct  gross_amount  net_amount                order_ts
   O1001         1      250.00          0.05        250.00     237.500 2026-09-01 09:15:00+00:00
   O1002         2       45.50          0.00         91.00      91.000 2026-09-01 10:10:00+00:00
   O1003         3       12.99          0.10         38.97      35.073 2026-09-01 11:05:00+00:00
   O1004         1       89.90          0.15         89.90      76.415 2026-09-02 08:45:00+00:00
   O1005         2       39.00          0.20         78.00      62.400 2026-09-02 12:00:00+00:00
```

Baru perintah ini yang benar-benar menjalankan file transformasi.

JALANKAN VERIFIKASI FILE PARQUET DENGAN MENGGUNAKAN PERINTAH INI

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> python -c "import pandas as pd; d=pd.read_parquet('data/processed/transformed_orders.parquet'); print('Jumlah record:',len(d)); print('Jumlah order unik:',d['order_id'].nunique()); print('Duplikasi order_id:',d['order_id'].duplicated().sum()); print('\nTipe data:'); print(d[['order_ts','quantity','unit_price','discount_pct','gross_amount','net_amount']].dtypes); print('\nContoh hasil:'); print(d[['order_id','gross_amount','net_amount']].head().to_string(index=False))"
Jumlah record: 30
Jumlah order unik: 30
Duplikasi order_id: 0

Tipe data:
order_ts        datetime64[us, UTC]
quantity                      int64
unit_price                  float64
discount_pct                float64
gross_amount                float64
net_amount                  float64
dtype: object

Contoh hasil:
order_id  gross_amount  net_amount
   O1001        250.00     237.500
   O1002         91.00      91.000
   O1003         38.97      35.073
   O1004         89.90      76.415
   O1005         78.00      62.400
```

Terlihat bahwa kolom order_ts bertipe data datetime dengan zona waktu UTC, kolom quantity bertipe interger, dan kolom unit_price, discount_pct, serta hasil perhitungan (gross_amount dan net_amount bertipe data numerik (float64).

WATERMARK DAN IDEMPOTENSI

BUAT FILE INCREMENTAL

scripts > incremental_upsert.py > ...

```python
"""Incremental upsert of transformed orders with a persistent checkpoint."""

import json
from pathlib import Path

import pandas as pd


SOURCE = Path("data/processed/transformed_orders.parquet")
TARGET = Path("data/warehouse/orders.parquet")
CHECKPOINT = Path("data/checkpoints/orders_watermark.json")
LOOKBACK_DAYS = 1


def main():
    if not SOURCE.exists():
        raise FileNotFoundError(f"Input tidak ditemukan: {SOURCE}")

    source_df = pd.read_parquet(SOURCE)
    source_df["order_ts"] = pd.to_datetime(
        source_df["order_ts"], utc=True, errors="raise"
    )

    if source_df["order_id"].isna().any():
        raise ValueError("order_id tidak boleh kosong.")

    if source_df["order_id"].duplicated().any():
        raise ValueError("Source memiliki order_id duplikat.")

```

```python
def main():
    old_watermark = None
    if CHECKPOINT.exists():
        checkpoint_data = json.loads(
            CHECKPOINT.read_text(encoding="utf-8")
        )
        old_watermark = pd.to_datetime(
            checkpoint_data["watermark"], utc=True
        )

    if old_watermark is None:
        # Eksekusi pertama: proses seluruh source.
        candidates = source_df.copy()
    else:
        # Baca ulang lookback window untuk mengakomodasi keterlambatan.
        cutoff = old_watermark - pd.Timedelta(days=LOOKBACK_DAYS)
        candidates = source_df.loc[
            source_df["order_ts"] >= cutoff
        ].copy()

    if TARGET.exists():
        existing_df = pd.read_parquet(TARGET)
        existing_df["order_ts"] = pd.to_datetime(
            existing_df["order_ts"], utc=True, errors="raise"
        )
    else:
        existing_df = pd.DataFrame(columns=source_df.columns)
```

```python
    # Upsert berdasarkan kunci bisnis order_id.
    combined = pd.concat(
        [existing_df, candidates],
        ignore_index=True,
    )
    combined = combined.drop_duplicates(
        subset=["order_id"], keep="last"
    )
    combined = combined.sort_values(
        "order_ts"
    ).reset_index(drop=True)

    TARGET.parent.mkdir(parents=True, exist_ok=True)
    CHECKPOINT.parent.mkdir(parents=True, exist_ok=True)

    # Simpan tujuan dahulu. Checkpoint baru ditulis setelah sukses.
    combined.to_parquet(TARGET, index=False)

    max_timestamp = source_df["order_ts"].max()
    if old_watermark is not None:
        max_timestamp = max(max_timestamp, old_watermark)

    checkpoint_data = {
        "watermark": max_timestamp.isoformat(),
        "lookback_days": LOOKBACK_DAYS,
        "business_key": "order_id",
    }
```

```python
    CHECKPOINT.write_text(
        json.dumps(checkpoint_data, indent=2),
        encoding="utf-8",
    )

    print("=== UPSERT SUMMARY ===")
    print(f"source_records={len(source_df)}")
    print(f"candidate_records={len(candidates)}")
    print(f"target_records={len(combined)}")
    print(f"unique_order_ids={combined['order_id'].nunique()}")
    print(f"duplicate_order_ids={combined['order_id'].duplicated().sum()}")
    print(f"previous_watermark={old_watermark}")
    print(f"new_watermark={max_timestamp.isoformat()}")
    print(f"target={TARGET}")
    print(f"checkpoint={CHECKPOINT}")


if __name__ == "__main__":
    main()
```

JALANKAN FILE INCREMENTAL TERSEBUT

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> python -m py_compile scripts/incremental_upsert.py
```

Checking adanya kesalahan penulisan syntax pada code

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> python -u scripts/incremental_upsert.py
=== UPSERT SUMMARY ===
source_records=30
candidate_records=30
target_records=30
unique_order_ids=30
duplicate_order_ids=0
previous_watermark=None
new_watermark=2026-09-10T16:30:00+00:00
target=data\warehouse\orders.parquet
checkpoint=data\checkpoints\orders_watermark.json
```

JALANKAN KEMBALI CODE INCREMENTAL TERSEBUT

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> python -u scripts/incremental_upsert.py
=== UPSERT SUMMARY ===
source_records=30
candidate_records=3
target_records=30
unique_order_ids=30
duplicate_order_ids=0
previous_watermark=2026-09-10 16:30:00+00:00
new_watermark=2026-09-10T16:30:00+00:00
target=data\warehouse\orders.parquet
checkpoint=data\checkpoints\orders_watermark.json
```

Karena proses menggunakan upsert berdasarkan `order_id`, jumlah target_records tetap sama apabila tidak ada order baru. Record pada lookback window akan dibaca ulang, tetapi tidak digandakan. Oleh karena itu juga, watermark masih sama karena tidak ditemukan watermark yang lebih besar (baru). Untuk membuktikan hal ini mari lakukan verifikasi ulang.

VERIFIKASI ULANG

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> python -c "import json,pandas as pd; d=pd.read_parquet('data/warehouse/orders.parquet'); w=json.load(open('data/checkpoints/orders_watermark.json',encoding='utf-8')); print('Jumlah record:',len(d)); print('Order unik:',d['order_id'].nunique()); print('Duplikasi:',d['order_id'].duplicated().sum()); print('Watermark:',w['watermark']); print('Checkpoint:',w)"
Jumlah record: 30
Order unik: 30
Duplikasi: 0
Watermark: 2026-09-10T16:30:00+00:00
Checkpoint: {'watermark': '2026-09-10T16:30:00+00:00', 'lookback_days': 1, 'business_key': 'order_id'}
```

ISI orders_watermark.json

data > checkpoints > {} orders_watermark.json > ...

```
{
  "watermark": "2026-09-10T16:30:00+00:00",
  "lookback_days": 1,
  "business_key": "order_id"
}
```

D. KAFKA STREAMING

BUAT FILE docker-compose.yml

docker-compose.yml

```yaml
services:
  kafka:
    image: apache/kafka:3.9.0
    container_name: dpw-kafka
    ports:
      - "9092:9092"
    environment:
      KAFKA_NODE_ID: 1
      KAFKA_PROCESS_ROLES: broker,controller
      KAFKA_LISTENERS: PLAINTEXT://:9092,CONTROLLER://:9093
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT
      KAFKA_CONTROLLER_QUORUM_VOTERS: 1@localhost:9093
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
      KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1
      KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 1
      KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS: 0
      KAFKA_NUM_PARTITIONS: 3
    healthcheck:
      test: ["CMD-SHELL", "/opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 --list >/dev/null 2>&1"]
      interval: 10s
      timeout: 5s
      retries: 10

```

BANGUN CONTAINER DENGAN MENJALANKAN PERINTAH BERIKUT :

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> docker compose up -d
[+] Running 11/11
 ✓ kafka Pulled                                                                  64.0s
   ✓ 43c4264eed91 Pull complete                                                  16.1s
   ✓ 21462eb7489f Pull complete                                                  23.8s
   ✓ 710b67c35e63 Pull complete                                                  47.3s
   ✓ d754a55bcb1a Pull complete                                                  47.1s
   ✓ acf463c2f208 Pull complete                                                  52.9s
   ✓ b8965cf5bd4c Pull complete                                                   1.9s
   ✓ 838d653125b3 Pull complete                                                  54.9s
   ✓ e706238dead8 Pull complete                                                  54.0s
   ✓ 5da4027ba785 Pull complete                                                  55.4s
   ✓ 068e70c969c9 Pull complete                                                  47.4s
[+] Running 2/2
 ✓ Network uts_default  Created                                                    0.2s
 ✓ Container dpw-kafka  Started                                                  12.7s
```

CEK KEBERADAAN CONTAINER DENGAN PERINTAH BERIKUT :

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> docker compose ps
NAME        IMAGE               COMMAND                  SERVICE   CREATED          STATUS                    PORTS
dpw-kafka   apache/kafka:3.9.0   "/__cacert_entrypoin…"   kafka     2 minutes ago    Up 2 minutes (healthy)    0.0.0.0:9092->9092/tcp
```

PERIKSA FILE DATA SUMBER

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> Get-ChildItem .\data -Recurse


    Directory: D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\data


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        10/25/2026  10:00 AM                checkpoints
d-----        10/25/2026   7:52 AM                processed
d-----        10/24/2026  10:51 PM                raw
d-----        10/25/2026  12:34 AM                raw_incremental_test
d-----        10/25/2026   1:03 AM                rejected
d-----        10/25/2026  10:00 AM                warehouse
-a----        10/24/2026   8:57 PM           2398 orders.csv
-a----        10/25/2026  12:32 AM           2511 orders_incremental_test.csv
-a----        10/25/2026  12:08 AM             48 watermark.json
-a----        10/25/2026  12:35 AM             48 watermark_incremental_test.json


    Directory: D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\data\checkpoints


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        10/25/2026  10:03 AM            102 orders_watermark.json
```

```
    Directory: D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\data\processed


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        10/25/2026   7:52 AM            472 transformation_summary.json
-a----        10/25/2026   7:52 AM           8131 transformed_orders.parquet
-a----        10/25/2026   1:03 AM            326 validation_summary.json
-a----        10/25/2026   1:03 AM           6538 valid_orders.parquet


    Directory: D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\data\raw


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        10/25/2026  12:08 AM           6770 orders.parquet


    Directory: D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\data\raw_incremental_test


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        10/25/2026  12:35 AM           6826 orders.parquet


    Directory: D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\data\rejected


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        10/25/2026   1:03 AM           2187 rejected_orders.jsonl
```

```
    Directory: D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\data\warehouse


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        10/25/2026  10:03 AM           8131 orders.parquet


(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS>
```

BUAT TOPIC BROKER (orders)

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> docker exec dpw-kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server
localhost:9092 --create --topic orders --partitions 3 --replication-factor 1
Created topic orders.
```

PERIKSA BROKER YANG DAPAT DI AKSES

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> docker exec dpw-kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server
localhost:9092 --list
orders
```

NOTE :

data yang digunakan adalah data asli untuk menguji consumer dalam memfilter data invalid dan valid

MEMBUAT FILE PROSEDUR

scripts > kafka_producer.py > ...

```python

import argparse
import csv
import json
from pathlib import Path

from kafka import KafkaProducer
from kafka.errors import KafkaError

PROJECT_DIR = Path(__file__).resolve().parent.parent
CSV_PATH = PROJECT_DIR / "data" / "orders.csv"
BOOTSTRAP_SERVERS = "localhost:9092"
TOPIC = "orders"


def main():
    parser = argparse.ArgumentParser(
        description="Mengirim event pesanan dari CSV ke Kafka."
    )
    parser.add_argument(
        "--limit",
        type=int,
        default=20,
        help="Jumlah baris CSV yang dikirim (default: 20).",
    )
    parser.add_argument(
        "--duplicate-first",
        action="store_true",
        help="Kirim ulang event pertama untuk demonstrasi duplikasi.",
    )
    args = parser.parse_args()

    if args.limit < 1:
        parser.error("--limit harus minimal 1")

    with CSV_PATH.open("r", encoding="utf-8-sig", newline="") as file:
        rows = list(csv.DictReader(file))

    if not rows:
        raise SystemExit("CSV tidak berisi data.")

    selected_rows = rows[:args.limit]

    producer = KafkaProducer(
        bootstrap_servers=BOOTSTRAP_SERVERS,
        key_serializer=lambda key: key.encode("utf-8"),
        value_serializer=lambda value: json.dumps(
            value, ensure_ascii=False
        ).encode("utf-8"),
        acks="all",
        retries=5,
    )

    sent = 0

    try:
        for row in selected_rows:
            order_id = row.get("order_id", "")
            if not order_id:
                print("Peringatan: event tanpa order_id akan memakai key kosong.")

            future = producer.send(
                TOPIC,
                key=order_id,
                value=row,
            )
            metadata = future.get(timeout=30)
            sent += 1
```

```
            print(
                f"SENT order_id={order_id} "
                f"topic={metadata.topic} "
                f"partition={metadata.partition} "
                f"offset={metadata.offset}"
            )

        if args.duplicate_first:
            row = selected_rows[0]
            order_id = row.get("order_id", "")

            metadata = producer.send(
                TOPIC,
                key=order_id,
                value=row,
            ).get(timeout=30)
            sent += 1

            print(
                f"DUPLICATE_SENT order_id={order_id} "
                f"topic={metadata.topic} "
                f"partition={metadata.partition} "
                f"offset={metadata.offset}"
            )

        producer.flush()
        print(f"Selesai. Total event terkirim: {sent}")

    except KafkaError as exc:
        raise SystemExit(f"Gagal mengirim event Kafka: {exc}") from exc
    finally:
        producer.close()


if __name__ == "__main__":
    main()
```

MEMBUAT FILE CONSUMER

scripts > kafka_consumer.py > ...

```python

import json
from datetime import datetime
from pathlib import Path
from typing import Literal

from kafka import KafkaConsumer
from kafka.structs import OffsetAndMetadata, TopicPartition
from pydantic import BaseModel, ConfigDict, Field, ValidationError


# ============================================================
# KONFIGURASI
# ============================================================

PROJECT_DIR = Path(__file__).resolve().parent.parent
OUTPUT_DIR = PROJECT_DIR / "data" / "streaming"

VALID_PATH = OUTPUT_DIR / "valid_orders.jsonl"
REJECTED_PATH = OUTPUT_DIR / "rejected_orders.jsonl"

BOOTSTRAP_SERVERS = "localhost:9092"
TOPIC = "orders"
GROUP_ID = "orders-consumer-v1"


# ============================================================
# KONTRAK DATA - SELARAS DENGAN TAHAP 2
# ============================================================

```

```python
class OrderContract(BaseModel):
    model_config = ConfigDict(extra="ignore")

    order_id: str = Field(min_length=1)
    customer_id: str = Field(min_length=1)
    order_ts: datetime

    product_category: str
    quantity: int = Field(gt=0)
    unit_price: float = Field(ge=0)
    discount_pct: float = Field(ge=0, le=1)

    payment_method: str
    status: Literal["paid", "shipped", "cancelled", "refunded"]


# ============================================================
# UTILITAS FILE JSONL
# ============================================================

def read_jsonl(path: Path) -> list[dict]:
    """Membaca file JSONL; file belum ada dianggap kosong."""
    if not path.exists():
        return []

    records = []
    with path.open("r", encoding="utf-8") as file:
        for line in file:
            if line.strip():
                records.append(json.loads(line))

    return records
```

```python
def write_jsonl_atomic(path: Path, records: list[dict]) -> None:
    """Menulis file sementara lalu mengganti file tujuan."""
    path.parent.mkdir(parents=True, exist_ok=True)
    temp_path = path.with_suffix(path.suffix + ".tmp")

    with temp_path.open("w", encoding="utf-8", newline="\n") as file:
        for record in records:
            file.write(
                json.dumps(record, ensure_ascii=False, default=str) + "\n"
            )
        file.flush()

    temp_path.replace(path)


# ============================================================
# SINK VALID: UPSERT BERDASARKAN order_id
# ============================================================

def save_valid(order: dict, kafka_metadata: dict) -> str:
    records = read_jsonl(VALID_PATH)

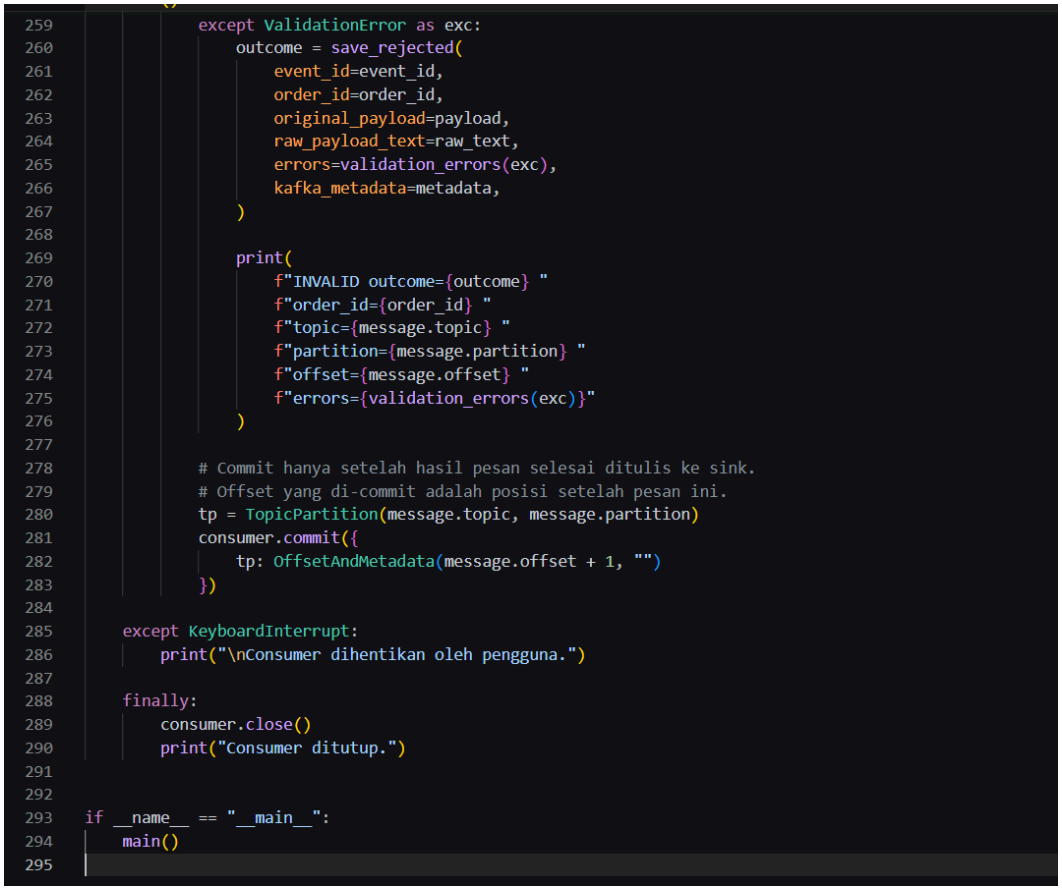
    # Metadata Kafka tidak dihitung sebagai isi bisnis pesanan.
    new_business_data = order
    existing_index = None

    for index, record in enumerate(records):
        if record.get("order_id") == order["order_id"]:
            existing_index = index
            break

    if existing_index is not None:
        existing = records[existing_index]
        old_business_data = {
            key: value for key, value in existing.items()
            if key != "_kafka"
        }
```

:

:



```
            except ValidationError as exc:
                outcome = save_rejected(
                    event_id=event_id,
                    order_id=order_id,
                    original_payload=payload,
                    raw_payload_text=raw_text,
                    errors=validation_errors(exc),
                    kafka_metadata=metadata,
                )

                print(
                    f"INVALID outcome={outcome} "
                    f"order_id={order_id} "
                    f"topic={message.topic} "
                    f"partition={message.partition} "
                    f"offset={message.offset} "
                    f"errors={validation_errors(exc)}"
                )

            # Commit hanya setelah hasil pesan selesai ditulis ke sink.
            # Offset yang di-commit adalah posisi setelah pesan ini.
            tp = TopicPartition(message.topic, message.partition)
            consumer.commit({
                tp: OffsetAndMetadata(message.offset + 1, "")
            })

    except KeyboardInterrupt:
        print("\nConsumer dihentikan oleh pengguna.")

    finally:
        consumer.close()
        print("Consumer ditutup.")


if __name__ == "__main__":
    main()
```

Validasi kontrak. Consumer memeriksa order_id, customer_id, order_ts, quantity, unit_price, discount_pct, dan status sesuai aturan Tahap 2. Contohnya, quantity harus lebih dari nol, discount_pct harus berada di antara 0 dan 1, dan status harus termasuk empat nilai yang diizinkan.

Pemisahan output. Record valid disimpan di data/streaming/valid_orders.jsonl. Record invalid dan detail kesalahannya disimpan di data/streaming/rejected_orders.jsonl, termasuk payload asli.

Idempotensi. Untuk record valid, order_id menjadi kunci upsert. Jika event identik diproses ulang, consumer mengabaikannya, bukan menambahkan record bisnis baru. Untuk rejected, identitas pesan Kafka (topic, partition, offset) dipakai agar replay pesan yang sama tidak menambah baris rejected baru.

Commit offset. Consumer melakukan commit setelah penulisan sink selesai. Ini memberikan pola at-least-once, bukan exactly-once end-to-end. Implementasi file ini juga ditujukan untuk praktikum satu consumer; pembaruan file secara atomik membantu mencegah file setengah tertulis, tetapi bukan transaksi bersama antara file dan Kafka.

MENJALANKAN FILE CONSUMER
BEFORE PRODUCER

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS>  python .\scripts\kafka_consumer.py
Consumer aktif untuk topic=orders
Broker=localhost:9092, group_id=orders-consumer-v1
File valid   : D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\data\streaming\valid_orders.jsonl
File rejected: D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\data\streaming\rejected_orders.jsonl
Menunggu pesan... Tekan Ctrl+C untuk berhenti.
```

AFTER PRODUCER

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS>  python .\scripts\kafka_consumer.py
File rejected: D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\data\streaming\rejected_orders.jsonl
Menunggu pesan... Tekan Ctrl+C untuk berhenti.
VALID outcome=DUPLICATE_IGNORED order_id=O1001 topic=orders partition=0 offset=14
VALID outcome=DUPLICATE_IGNORED order_id=O1002 topic=orders partition=2 offset=20
VALID outcome=DUPLICATE_IGNORED order_id=O1003 topic=orders partition=2 offset=21
VALID outcome=DUPLICATE_IGNORED order_id=O1004 topic=orders partition=0 offset=15
VALID outcome=DUPLICATE_IGNORED order_id=O1007 topic=orders partition=1 offset=6
VALID outcome=DUPLICATE_IGNORED order_id=O1005 topic=orders partition=2 offset=22
VALID outcome=DUPLICATE_IGNORED order_id=O1006 topic=orders partition=2 offset=23
VALID outcome=DUPLICATE_IGNORED order_id=O1008 topic=orders partition=0 offset=16
VALID outcome=DUPLICATE_IGNORED order_id=O1009 topic=orders partition=1 offset=7
VALID outcome=DUPLICATE_IGNORED order_id=O1012 topic=orders partition=1 offset=8
VALID outcome=DUPLICATE_IGNORED order_id=O1010 topic=orders partition=2 offset=24
VALID outcome=DUPLICATE_IGNORED order_id=O1013 topic=orders partition=2 offset=25
VALID outcome=DUPLICATE_IGNORED order_id=O1015 topic=orders partition=2 offset=26
VALID outcome=DUPLICATE_IGNORED order_id=O1011 topic=orders partition=0 offset=17
VALID outcome=DUPLICATE_IGNORED order_id=O1014 topic=orders partition=0 offset=18
VALID outcome=INSERTED order_id=O1021 topic=orders partition=1 offset=9
VALID outcome=DUPLICATE_IGNORED order_id=O1018 topic=orders partition=2 offset=27
VALID outcome=DUPLICATE_IGNORED order_id=O1019 topic=orders partition=2 offset=28
VALID outcome=DUPLICATE_IGNORED order_id=O1020 topic=orders partition=2 offset=29
VALID outcome=INSERTED order_id=O1023 topic=orders partition=2 offset=30
VALID outcome=DUPLICATE_IGNORED order_id=O1016 topic=orders partition=0 offset=19
VALID outcome=DUPLICATE_IGNORED order_id=O1017 topic=orders partition=0 offset=20
VALID outcome=INSERTED order_id=O1022 topic=orders partition=0 offset=21
INVALID outcome=REJECTED_SAVED order_id=O1032 topic=orders partition=1 offset=10 errors=[{'field': 'unit_price',
'message': 'Input should be greater than or equal to 0', 'type': 'greater_than_equal'}]
VALID outcome=INSERTED order_id=O1024 topic=orders partition=2 offset=31
VALID outcome=INSERTED order_id=O1028 topic=orders partition=2 offset=32
VALID outcome=INSERTED order_id=O1030 topic=orders partition=2 offset=33
INVALID outcome=REJECTED_SAVED order_id=O1031 topic=orders partition=2 offset=34 errors=[{'field': 'quantity', 'm
essage': 'Input should be greater than 0', 'type': 'greater_than'}]
INVALID outcome=REJECTED_SAVED order_id=O1034 topic=orders partition=2 offset=35 errors=[{'field': 'order_ts', 'm
essage': 'Input should be a valid datetime or date, invalid character in year', 'type': 'datetime_from_date_parsi
ng'}]
VALID outcome=INSERTED order_id=O1025 topic=orders partition=0 offset=22
VALID outcome=INSERTED order_id=O1026 topic=orders partition=0 offset=23
VALID outcome=INSERTED order_id=O1027 topic=orders partition=0 offset=24
VALID outcome=INSERTED order_id=O1029 topic=orders partition=0 offset=25
INVALID outcome=REJECTED_SAVED order_id=O1033 topic=orders partition=0 offset=26 errors=[{'field': 'discount_pct'
, 'message': 'Input should be less than or equal to 1', 'type': 'less_than_equal'}]
INVALID outcome=REJECTED_SAVED order_id=O1035 topic=orders partition=0 offset=27 errors=[{'field': 'status', 'mes
sage': "Input should be 'paid', 'shipped', 'cancelled' or 'refunded'", 'type': 'literal_error'}]
INVALID outcome=REJECTED_SAVED order_id=O1036 topic=orders partition=0 offset=28 errors=[{'field': 'customer_id',
 'message': 'String should have at least 1 character', 'type': 'string_too_short'}]
```

MENJALANKAN PRODUCER

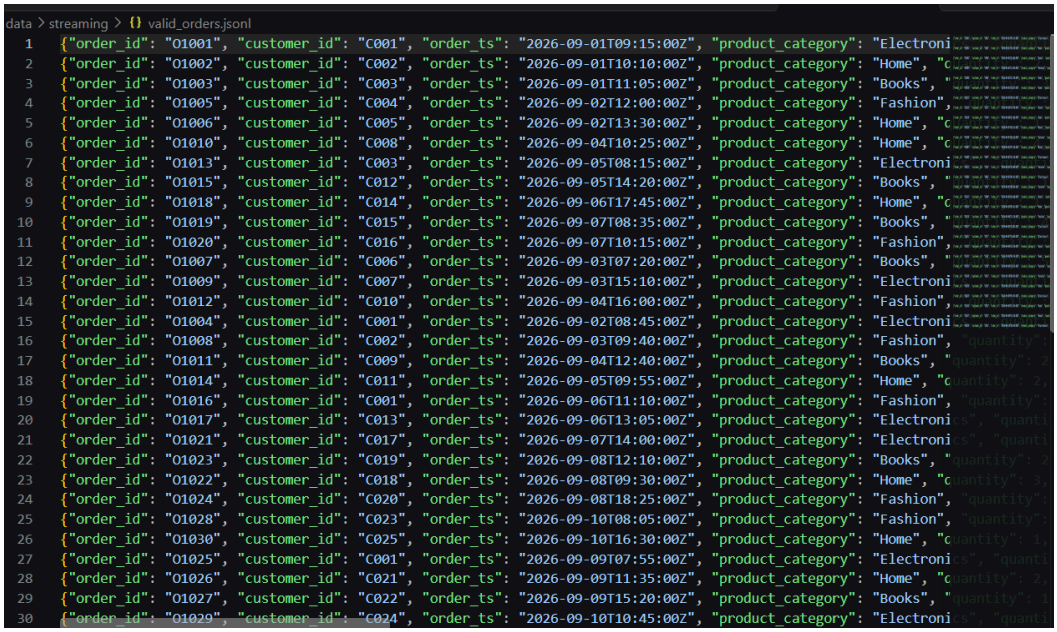
```
Selesai. Total event terkirim: 20
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> python .\scripts\kafka_producer.py
D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\scripts\kafka_producer.py:105: DeprecationWarning: key_seri
alizer does not implement kafka.serializer.Serializer
  main()
D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\scripts\kafka_producer.py:105: DeprecationWarning: value_s
rializer does not implement kafka.serializer.Serializer
  main()
SENT order_id=O1001 topic=orders partition=0 offset=14
SENT order_id=O1002 topic=orders partition=2 offset=20
SENT order_id=O1003 topic=orders partition=2 offset=21
SENT order_id=O1004 topic=orders partition=0 offset=15
SENT order_id=O1005 topic=orders partition=2 offset=22
SENT order_id=O1006 topic=orders partition=2 offset=23
SENT order_id=O1007 topic=orders partition=1 offset=6
SENT order_id=O1008 topic=orders partition=0 offset=16
SENT order_id=O1009 topic=orders partition=1 offset=7
SENT order_id=O1010 topic=orders partition=2 offset=24
SENT order_id=O1011 topic=orders partition=0 offset=17
SENT order_id=O1012 topic=orders partition=1 offset=8
SENT order_id=O1013 topic=orders partition=2 offset=25
SENT order_id=O1014 topic=orders partition=0 offset=18
SENT order_id=O1015 topic=orders partition=2 offset=26
SENT order_id=O1016 topic=orders partition=0 offset=19
SENT order_id=O1017 topic=orders partition=0 offset=20
SENT order_id=O1018 topic=orders partition=2 offset=27
SENT order_id=O1019 topic=orders partition=2 offset=28
SENT order_id=O1020 topic=orders partition=2 offset=29
SENT order_id=O1021 topic=orders partition=1 offset=9
SENT order_id=O1022 topic=orders partition=0 offset=21
SENT order_id=O1023 topic=orders partition=2 offset=30
SENT order_id=O1024 topic=orders partition=2 offset=31
SENT order_id=O1025 topic=orders partition=0 offset=22
SENT order_id=O1026 topic=orders partition=0 offset=23
SENT order_id=O1027 topic=orders partition=0 offset=24
SENT order_id=O1028 topic=orders partition=2 offset=32
SENT order_id=O1029 topic=orders partition=0 offset=25
SENT order_id=O1030 topic=orders partition=2 offset=33
SENT order_id=O1031 topic=orders partition=2 offset=34
SENT order_id=O1032 topic=orders partition=1 offset=10
SENT order_id=O1033 topic=orders partition=0 offset=26
SENT order_id=O1034 topic=orders partition=2 offset=35
SENT order_id=O1035 topic=orders partition=0 offset=27
SENT order_id=O1036 topic=orders partition=0 offset=28
Selesai. Total event terkirim: 36
```

DATA YANG MASUK INVALID DATA (REJECT)

data > streaming > {} rejected_orders.jsonl

```
{"_event_id": "orders:1:10", "order_id": "O1032", "original_payload": {"order_id": "O1032", "customer_id": "C02
{"_event_id": "orders:2:34", "order_id": "O1031", "original_payload": {"order_id": "O1031", "customer_id": "C02
{"_event_id": "orders:2:35", "order_id": "O1034", "original_payload": {"order_id": "O1034", "customer_id": "C02
{"_event_id": "orders:0:26", "order_id": "O1033", "original_payload": {"order_id": "O1033", "customer_id": "C02
{"_event_id": "orders:0:27", "order_id": "O1035", "original_payload": {"order_id": "O1035", "customer_id": "C03
{"_event_id": "orders:0:28", "order_id": "O1036", "original_payload": {"order_id": "O1036", "customer_id": "",
```

DATA YANG MASUK VALID DATA (VALID)



VERIFIKASI JUMLAH DATA VALID DAN TIDAK VALID (REJECT)

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> if (Test-Path .\data\streaming\valid_orders.j
nl) {
>>     "Valid: $((Get-Content .\data\streaming\valid_orders.jsonl | Measure-Object -Line).Lines)"
>> }
Valid: 30
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> if (Test-Path .\data\streaming\rejected_order
jsonl) {
>>     "Rejected: $((Get-Content .\data\streaming\rejected_orders.jsonl | Measure-Object -Line).Lines)"
>> }
Rejected: 6
```

DEMONSTRASI DATA DUPLIKAT

Producer

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> python .\scripts\kafka_producer.py --limit 1 --
duplicate-first
D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\scripts\kafka_producer.py:105: DeprecationWarning: key_seri
alizer does not implement kafka.serializer.Serializer
  main()
D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\scripts\kafka_producer.py:105: DeprecationWarning: value_se
rializer does not implement kafka.serializer.Serializer
  main()
SENT order_id=O1001 topic=orders partition=0 offset=29
DUPLICATE_SENT order_id=O1001 topic=orders partition=0 offset=30
Selesai. Total event terkirim: 2
```

Consumer

```
Rejected: 6
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS>  python .\scripts\kafka_consumer.py
Consumer aktif untuk topic=orders
Broker=localhost:9092, group_id=orders-consumer-v1
File valid    : D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\data\streaming\valid_orders.jsonl
File rejected: D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\data\streaming\rejected_orders.jsonl
Menunggu pesan... Tekan Ctrl+C untuk berhenti.
VALID outcome=DUPLICATE_IGNORED order_id=O1001 topic=orders partition=0 offset=29
VALID outcome=DUPLICATE_IGNORED order_id=O1001 topic=orders partition=0 offset=30
```

Verifikasi jumlah data

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> (Get-Content .\data\streaming\valid_orders.json
l | Measure-Object -Line).Lines
30
```

DESKRIPSI CONSUMER

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> docker exec dpw-kafka /opt/kafka/bin/kafka-cons
umer-groups.sh --bootstrap-server localhost:9092 --describe --group orders-consumer-v1

Consumer group 'orders-consumer-v1' has no active members.

GROUP              TOPIC          PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG             CONSUMER-ID     HOS
T           CLIENT-ID
orders-consumer-v1 orders         0          31              31              0               -               -
           -
orders-consumer-v1 orders         1          11              11              0               -               -
           -
orders-consumer-v1 orders         2          36              36              0               -               -
           -
```

DEFINISI “At-most-once”

At-most-once adalah pesan yang diproses max satu kali. Jika offset di-comit sebelum data berhasil disimpan, laly program gagal, pesan tersebut bisa terlewat saat consumer melanjutkan proses.

DEFINISI “At-least-once”

Consumer menulis data ke sink terlebih dahulu, kemudian melakukan commit offset. Apabila program mengalami kegagalan setelah sink berhasil ditulis tetapi sebelum offset di-commit maka pesan tersebut akan dapat dikonsumsi kembali.

DEFINISI “ Exactly-once “

Adalah setiap event yang menghasilkan efek bisnis tepat satu kali, termasuk ketika terjadi kegagalan dan pemrosesan ulang. Untuk menjaminnya, mekanisme harus mencakup batas antara pemrosesan pesan, penyimpanan data, dan pengelolaan offset yang relevan.

FUNGSI DARI :

1. **CONSUMER GROUPS** yang digunakan adalah **consumer group orders-consumer-v1.** Kemudian fungsi dari consumer group itu sendiri adalah untuk membagi pekerjaan konsumsi pesan Kafka ke beberapa consumer agar pesan dari sebuah topic dapat diproses secara paralel. Selain itu consumer group juga menyimpan progres konsumsi melalui offset yang dicommit sehingga consumer dapat melanjutkan dari posisi terakhir yang tercatat.
2. **PARTITIONS** adalah untuk membagi topic menjadi beberapa urutan log yang memungkinkan pemrosesan secara paralel terjadi, tetapi partition tidak ada jaminan secara global antar partition.
3. **OFFSET** adalah untuk menunjukkan posisi pesan dalam sebuah partition dan digunakan untuk melacak kemajuan konsumsi.

PERBEDAAN At-least-once dan Exactly-once :

At-least-once menjamin pesan dapat diproses satu kali atau lebih, sehingga pemrosesan ulang masih mungkin terjadi apabila consumer gagal sebelum offset berhasil di-commit. Sementara itu, exactly-once bertujuan memastikan setiap event menghasilkan efek yang relevan tepat satu kali, termasuk ketika terjadi kegagalan dan pemrosesan ulang.

E. BENCHMARK PANDAS DAN POLAR

MEMBUAT FOLDER DATA HASIL BENCHMARK

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> New-Item -ItemType Directory -Force .\data\benc
hmark


    Directory: D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\data


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        10/25/2026   8:10 PM                benchmark
```

MEMBUAT FILE benchmark_egines.py

scripts > benchmark_engines.py > ...

```python

import csv
import math
import os
import platform
import statistics
import time
from pathlib import Path

import pandas as pd
import polars as pl


# Lokasi proyek dan dataset
ROOT = Path(__file__).resolve().parent.parent
DATA_PATH = ROOT / "data" / "raw" / "orders.parquet"
OUTPUT_PATH = ROOT / "data" / "benchmark" / "benchmark_results.csv"

# Jumlah pengulangan pengukuran, setelah warm-up
REPEATS = 10


def benchmark_pandas():
    start = time.perf_counter()

    df = pd.read_parquet(DATA_PATH)

    df["gross_amount"] = df["quantity"] * df["unit_price"]
    df["net_amount"] = (
        df["gross_amount"] * (1 - df["discount_pct"])
    )

    result = (
        df.groupby("product_category", dropna=False)["net_amount"]
        .sum()
        .reset_index()
    )
```

```
    elapsed = time.perf_counter() - start
    return elapsed, result


def benchmark_polars_eager():
    start = time.perf_counter()

    df = pl.read_parquet(DATA_PATH)

    df = df.with_columns(
        (pl.col("quantity") * pl.col("unit_price"))
        .alias("gross_amount"),
        (
            pl.col("quantity")
            * pl.col("unit_price")
            * (1 - pl.col("discount_pct"))
        ).alias("net_amount"),
    )

    result = (
        df.group_by("product_category")
        .agg(pl.col("net_amount").sum())
    )

    elapsed = time.perf_counter() - start
    return elapsed, result


def benchmark_polars_lazy():
    start = time.perf_counter()
```

```
    result = (
        pl.scan_parquet(str(DATA_PATH))
        .with_columns(
            (pl.col("quantity") * pl.col("unit_price"))
            .alias("gross_amount"),
            (
                pl.col("quantity")
                * pl.col("unit_price")
                * (1 - pl.col("discount_pct"))
            ).alias("net_amount"),
        )
        .group_by("product_category")
        .agg(pl.col("net_amount").sum())
        .collect()
    )

    elapsed = time.perf_counter() - start
    return elapsed, result


def normalize_result(result, engine):
    """Ubah hasil agregasi menjadi dictionary untuk pemeriksaan."""
    if engine == "pandas":
        records = result.to_dict("records")
    else:
        records = result.to_dicts()

    normalized = {}

    for row in records:
        category = row["product_category"]
        key = "<NULL>" if category is None or pd.isna(category) else str(category)
        value = float(row["net_amount"])
        normalized[key] = value
```

:

:

```python
    # Ringkasan statistik
    rows = []

    print("\n=== HASIL BENCHMARK ===")
    print(f"{'Engine':<16} {'Median (ms)':>14} {'Rata-rata (ms)':>17}")

    for name, values in timings.items():
        median_ms = statistics.median(values) * 1000
        mean_ms = statistics.mean(values) * 1000

        print(f"{name:<16} {median_ms:>14.3f} {mean_ms:>17.3f}")

        rows.append({
            "engine": name,
            "dataset": str(DATA_PATH.relative_to(ROOT)),
            "rows": len(df_info),
            "columns": len(df_info.columns),
            "file_size_bytes": os.path.getsize(DATA_PATH),
            "repeats": REPEATS,
            "median_ms": round(median_ms, 6),
            "mean_ms": round(mean_ms, 6),
        })

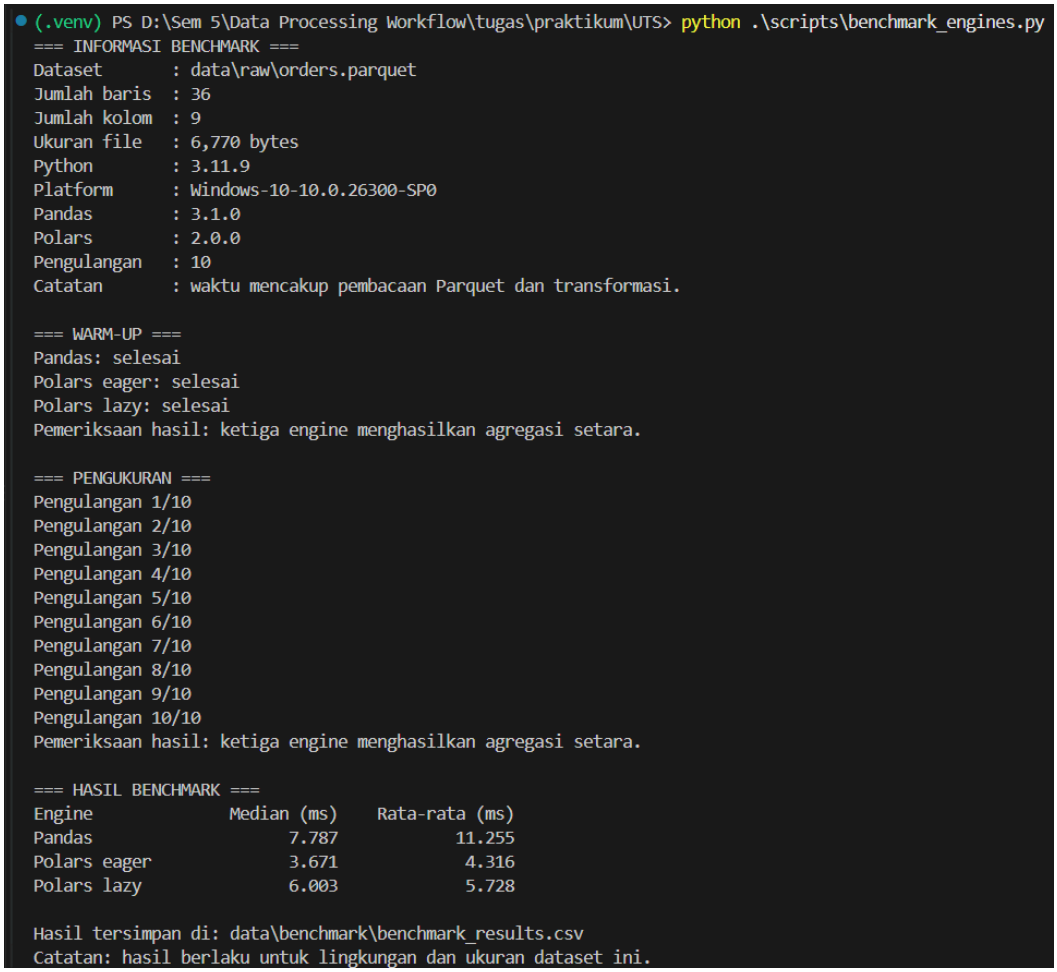
    # Simpan hasil agar dapat dipakai dalam laporan
    with OUTPUT_PATH.open("w", newline="", encoding="utf-8") as file:
        writer = csv.DictWriter(file, fieldnames=rows[0].keys())
        writer.writeheader()
        writer.writerows(rows)

    print(f"\nHasil tersimpan di: {OUTPUT_PATH.relative_to(ROOT)}")
    print("Catatan: hasil berlaku untuk lingkungan dan ukuran dataset ini.")


if __name__ == "__main__":
    main()

```

JALANKAN FILE benchmark_engine.py



Berdasarkan hasil benchmark_engine diatas terlihat bahwa Polar eager paling cepat dengan rata-rata eksekusi 4.3 milidetik dalam pengujian benchmark, disusul dengan Polar Lazy dengan rata-rata waktu eksekusi 5.7 milidetik, kemudian baru Pandas, dengan rata-rata waktu eksekusi yaitu 11.25 milidetitik.

Alasan yang mendasari Polar Lazy belum menjadi metode tercepat dalam pengujian ini, salah satu kemungkinannya adalh ukuran dataset yang sangat kecil sehingga manfaat optimasi rencana eksekusi belum cukup untuk mengimbangi overhead pemrosesan. Oleh karena itu, hasil benchmark ini hanya menggambarkan performa pada dataset dan lingkungan pengujian yang digunakan, bukan membuktikan bahwa satu library selalu lebih unggul daripada semua skala data.

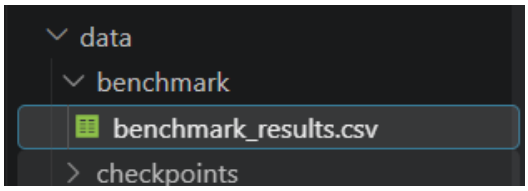
Perlu diperhatikan juga bahwa dataset yang digunakan merupakan data raw yang juga memiliki data record yang gagal validasi. Dengan demikian, eksperimen ini ditunjukkan untuk membandingkan performa komputasi, bukan untuk menghasilkan laporan yang telah tervalidasi.

HASIL PERBANDINGAN INI TELAH TERSIMPAN DALAM FILE CSV BERIKUT :

data > benchmark > benchmark_results.csv

```
engine,dataset,rows,columns,file_size_bytes,repeats,median_ms,mean_ms
Pandas,data\raw\orders.parquet,36,9,6770,10,7.7867,11.25521
Polars eager,data\raw\orders.parquet,36,9,6770,10,3.67085,4.31638
Polars lazy,data\raw\orders.parquet,36,9,6770,10,6.0031,5.72809

```



EKSPERIMEN DATA SKEW

Membuat file benchmark_skew.py

scripts > benchmark_skew.py > ...

```python

from pathlib import Path
from time import perf_counter
import platform
import statistics

import pandas as pd
import polars as pl


# ================================================
# 1. KONFIGURASI
# ================================================
ROOT = Path(__file__).resolve().parents[1]
SOURCE = ROOT / "data" / "raw" / "orders.parquet"
OUTPUT_DIR = ROOT / "data" / "benchmark"

OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

REPEATS_DATA = 100
SKEW_CUSTOMER = "C_SKEW_DOMINANT"
BENCHMARK_RUNS = 10


# ================================================
# 2. BACA DATA DAN BUAT DATASET NORMAL + SKEWED
# ================================================
base_df = pd.read_parquet(SOURCE)

normal_parts = []
skewed_parts = []
```

```python
for i in range(REPEATS_DATA):
    # Salinan untuk dataset normal
    normal_part = base_df.copy()
    normal_part["order_id"] = (
        normal_part["order_id"].astype(str) + f"_R{i + 1:03d}"
    )
    normal_parts.append(normal_part)

    # Salinan untuk dataset skewed
    skewed_part = base_df.copy()
    skewed_part["order_id"] = (
        skewed_part["order_id"].astype(str) + f"_R{i + 1:03d}"
    )

    # 99 dari 100 salinan menggunakan customer_id yang sama.
    # Salinan terakhir mempertahankan customer_id asli.
    if i < REPEATS_DATA - 1:
        skewed_part["customer_id"] = SKEW_CUSTOMER

    skewed_parts.append(skewed_part)

normal_df = pd.concat(normal_parts, ignore_index=True)
skewed_df = pd.concat(skewed_parts, ignore_index=True)

normal_path = OUTPUT_DIR / "orders_expanded_normal.parquet"
skewed_path = OUTPUT_DIR / "orders_expanded_skewed.parquet"

normal_df.to_parquet(normal_path, index=False)
skewed_df.to_parquet(skewed_path, index=False)

# Kedua dataset harus memiliki jumlah baris yang sama.
assert len(normal_df) == len(skewed_df)

# Hitung nilai net_amount untuk eksperimen agregasi.
for df in (normal_df, skewed_df):
    df["gross_amount"] = df["quantity"] * df["unit_price"]
    df["net_amount"] = df["gross_amount"] * (1 - df["discount_pct"])
```

```python
# ================================================
# 3. ANALISIS DISTRIBUSI CUSTOMER_ID
# ================================================
distribution_parts = []

for dataset_name, df in [
    ("normal", normal_df),
    ("skewed", skewed_df),
]:
    counts = (
        df.groupby("customer_id", dropna=False)
        .size()
        .reset_index(name="row_count")
        .sort_values("row_count", ascending=False)
    )

    counts["dataset"] = dataset_name
    counts["percentage"] = counts["row_count"] / len(df) * 100

    distribution_parts.append(counts)

distribution = pd.concat(distribution_parts, ignore_index=True)

distribution_path = OUTPUT_DIR / "skew_customer_distribution.csv"
distribution.to_csv(distribution_path, index=False)
```

```python
# ================================================
# 4. FUNGSI AGREGASI
# ================================================
def pandas_aggregate(df):
    return (
        df.groupby("customer_id", dropna=False)
        .agg(
            order_count=("order_id", "count"),
            total_net_amount=("net_amount", "sum"),
        )
        .reset_index()
    )


def polars_eager_aggregate(df):
    return df.group_by("customer_id").agg(
        pl.len().alias("order_count"),
        pl.col("net_amount").sum().alias("total_net_amount"),
    )


def polars_lazy_aggregate(df):
    return (
        df.lazy()
        .group_by("customer_id")
        .agg(
            pl.len().alias("order_count"),
            pl.col("net_amount").sum().alias("total_net_amount"),
        )
        .collect()
    )

```

:

:

```python
print("=" * 65)
print("DATA SKEW EXPERIMENT")
print("=" * 65)
print("Python:", platform.python_version())
print("Pandas:", pd.__version__)
print("Polars:", pl.__version__)
print("Dataset sumber:", SOURCE)
print("Baris dataset normal:", len(normal_df))
print("Baris dataset skewed:", len(skewed_df))

print("\nDistribusi customer_id (10 teratas per dataset):")
print(
    distribution.groupby("dataset", group_keys=False)
    .head(10)[["dataset", "customer_id", "row_count", "percentage"]]
    .to_string(index=False)
)

print("\nHasil benchmark:")
print(
    results_df[
        [
            "dataset",
            "engine",
            "rows",
            "customer_groups",
            "median_ms",
            "mean_ms",
            "estimated_memory_bytes",
        ]
    ].to_string(index=False)
)

print("\nFile hasil:")
print("Normal dataset :", normal_path)
print("Skewed dataset :", skewed_path)
print("Distribusi     :", distribution_path)
print("Benchmark      :", results_path)
```

JALANKAN SKEW

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> python .\scripts\benchmark_skew.py
============================================================
DATA SKEW EXPERIMENT
============================================================
Python: 3.11.9
Pandas: 3.1.0
Polars: 2.0.0
Dataset sumber: D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\data\raw\orders.parquet
Baris dataset normal: 3600
Baris dataset skewed: 3600

Distribusi customer_id (10 teratas per dataset):
dataset     customer_id  row_count  percentage
 normal            C001        400   11.111111
 normal            C002        200    5.555556
 normal            C003        200    5.555556
 normal            C004        100    2.777778
 normal            C005        100    2.777778
 normal            C006        100    2.777778
 normal            C007        100    2.777778
 normal            C008        100    2.777778
 normal            C009        100    2.777778
 normal            C010        100    2.777778
 skewed C_SKEW_DOMINANT       3564   99.000000
 skewed            C001          4    0.111111
 skewed            C003          2    0.055556
 skewed            C002          2    0.055556
 skewed            C005          1    0.027778
 skewed            C006          1    0.027778
 skewed            C007          1    0.027778
 skewed            C004          1    0.027778
 skewed            C009          1    0.027778
 skewed            C010          1    0.027778
```

```
Hasil benchmark:
dataset       engine  rows  customer_groups  median_ms  mean_ms  estimated_memory_bytes
 normal       Pandas  3600               31   31.34095 38.34330                  430732
 normal Polars eager  3600               31    1.63905  2.65607                  286550
 normal  Polars lazy  3600               31    1.89900  2.29197                  286550
 skewed       Pandas  3600               32   32.40395 32.22994                  470332
 skewed Polars eager  3600               32    1.21790  2.16487                  326150
 skewed  Polars lazy  3600               32    1.78010  2.77995                  326150

File hasil:
Normal dataset : D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\data\benchmark\orders_expanded_normal.parq
uet
Skewed dataset : D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\data\benchmark\orders_expanded_skewed.parq
uet
Distribusi     : D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\data\benchmark\skew_customer_distribution.
csv
Benchmark      : D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\data\benchmark\skew_benchmark_results.csv
```

DAMPAK DATA SKEW PADA OPERASI TERDISTRIBUSI

Data skew merupakan kondisi ketika distribusi data tidak merata sehingga satu atau beberapa grup memiliki jumlah data yang jauh lebih besar dibandingkan grup lainnya. Pada operasi agregasi berdasarkan kunci tertentu, seperti `GROUP BY customer_id`, ketimpangan tersebut dapat menyebabkan beban pemrosesan tidak terbagi secara merata di antara worker dalam sistem terdistribusi.

Worker yang menerima grup dengan jumlah data terbesar membutuhkan waktu dan sumber daya lebih banyak untuk menyelesaikan pemrosesan. Sementara itu, worker lain

mungkin telah menyelesaikan tugasnya lebih awal. Kondisi ini dapat menimbulkan bottleneck dan meningkatkan waktu penyelesaian keseluruhan proses.

Pada eksperimen ini, sebanyak 3.564 dari 3.600 baris pada dataset skewed menggunakan `customer_id` bernilai `C_SKEW_DOMINANT`, atau sebesar 99% dari total data. Hasil tersebut menunjukkan konsentrasi data yang tinggi pada satu kunci. Meskipun eksperimen dilakukan pada satu komputer dan tidak mengukur pembagian beban antar-worker terdistribusi secara langsung, distribusi tersebut menggambarkan kondisi yang berpotensi menimbulkan ketimpangan beban pada sistem terdistribusi.

STRATEGI MITIGASI DATA SKEW

Beberapa strategi dapat digunakan untuk mengurangi dampak data skew terhadap efisiensi pemrosesan data. Pertama, partial aggregation dilakukan dengan menghitung hasil sementara pada setiap partisi sebelum menggabungkannya menjadi hasil akhir. Pendekatan ini dapat mengurangi jumlah data yang perlu dipertukarkan atau diproses pada tahap agregasi akhir.

Kedua, salting key dapat diterapkan ketika satu kunci mendominasi distribusi data. Kunci tersebut diberi nilai pembeda tambahan agar data dapat dibagi ke beberapa kelompok pemrosesan. Hasil agregasi dari kelompok-kelompok tersebut kemudian digabungkan kembali berdasarkan kunci aslinya.

Ketiga, pemrosesan per partisi atau batch dapat membantu mengendalikan penggunaan memori dan mengelola pemrosesan data berukuran besar secara bertahap. Keempat, hanya kolom yang diperlukan untuk analisis yang sebaiknya dibaca dan diproses agar penggunaan memori dan biaya pemrosesan dapat dikurangi.

Terakhir, pada Polars Lazy API, pemanggilan `collect()` dan materialisasi hasil sebaiknya dilakukan hanya ketika diperlukan. Penggabungan beberapa operasi dalam satu rencana lazy dapat memberikan kesempatan bagi optimizer untuk menyederhanakan pemrosesan dan menghindari eksekusi berulang yang tidak perlu.

Pemilihan strategi harus disesuaikan dengan penyebab masalah. Partial aggregation dan salting key lebih berkaitan langsung dengan penanganan ketimpangan beban agregasi, sedangkan pemrosesan batch, pemilihan kolom, dan pengendalian materialisasi terutama membantu efisiensi penggunaan sumber daya. Tidak semua strategi secara langsung menghilangkan data skew.

F. DUCKDB DAN DBT

MEMBUAT DATABASE DUCKDB DAN MEMUAT DATA VALID KE DALAMNYA
Membuat file setup_duckdb.py

scripts > setup_duckdb.py > ...

```python

from pathlib import Path

import duckdb


# Lokasi proyek dan file sumber
ROOT = Path(__file__).resolve().parents[1]
DB_PATH = ROOT / "analytics.duckdb"
SOURCE = ROOT / "data" / "processed" / "transformed_orders.parquet"

if not SOURCE.exists():
    raise FileNotFoundError(f"File sumber tidak ditemukan: {SOURCE}")

# Hubungkan ke database DuckDB.
# Database akan dibuat otomatis jika belum ada.
con = duckdb.connect(str(DB_PATH))

try:
    # Siapkan schema dan tabel dari data valid hasil transformasi.
    con.execute("CREATE SCHEMA IF NOT EXISTS raw")

    con.execute(
        """
        CREATE OR REPLACE TABLE raw.raw_orders AS
        SELECT *
        FROM read_parquet(?)
        """,
        [str(SOURCE)],
    )

    # Periksa jumlah baris dan keunikan order_id.
    total_rows = con.execute(
        "SELECT COUNT(*) FROM raw.raw_orders"
    ).fetchone()[0]

    unique_orders = con.execute(
        "SELECT COUNT(DISTINCT order_id) FROM raw.raw_orders"
    ).fetchone()[0]

    print("=== DUCKDB SETUP ===")
    print(f"Database       : {DB_PATH}")
    print(f"Tabel          : raw.raw_orders")
    print(f"Sumber         : {SOURCE}")
    print(f"Jumlah baris   : {total_rows}")
    print(f"Order unik     : {unique_orders}")

    # Tampilkan struktur tabel.
    print("\n=== STRUKTUR TABEL ===")
    schema_info = con.execute(
        "DESCRIBE raw.raw_orders"
    ).fetchdf()
    print(schema_info.to_string(index=False))

```

```
    # Tampilkan contoh data.
    print("\n=== 5 BARIS PERTAMA ===")
    sample = con.execute(
        """
        SELECT order_id, customer_id, order_ts,
               product_category, quantity, net_amount, status
        FROM raw.raw_orders
        ORDER BY order_id
        LIMIT 5
        """
    ).fetchdf()
    print(sample.to_string(index=False))

    # Pastikan hasil pemuatan sesuai dengan sumber.
    if total_rows != 30:
        raise ValueError(
            f"Jumlah baris tidak sesuai: diharapkan 30, ditemukan {total_rows}"
        )

    if unique_orders != total_rows:
        raise ValueError("Ditemukan order_id yang tidak unik!")

    print("\nPemeriksaan berhasil: seluruh baris berasal dari data valid.")

finally:
    con.close()

```

Jalankan

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> python .\scripts\setup_duckdb.py
=== DUCKDB SETUP ===
Database       : D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\analytics.duckdb
Tabel          : raw.raw_orders
Sumber         : D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\data\processed\transformed_orders.parquet
Jumlah baris   : 30
Order unik     : 30

=== STRUKTUR TABEL ===
     column_name              column_type null  key default extra
        order_id                  VARCHAR  YES None    None  None
     customer_id                  VARCHAR  YES None    None  None
        order_ts TIMESTAMP WITH TIME ZONE  YES None    None  None
        quantity                   BIGINT  YES None    None  None
      unit_price                   DOUBLE  YES None    None  None
    discount_pct                   DOUBLE  YES None    None  None
          status                  VARCHAR  YES None    None  None
product_category                  VARCHAR  YES None    None  None
  payment_method                  VARCHAR  YES None    None  None
    gross_amount                   DOUBLE  YES None    None  None
      net_amount                   DOUBLE  YES None    None  None

=== 5 BARIS PERTAMA ===
D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\scripts\setup_duckdb.py:65: Pandas4Warning: The inplace key
word in DataFrame.drop is deprecated and will be removed in a future version (PDEP-8).
See https://pandas.pydata.org/docs/dev/whatsnew/v3.1.0.html#deprecation-inplace for more details.
  ).fetchdf()
order_id customer_id                  order_ts product_category  quantity  net_amount    status
   O1001        C001 2026-09-01 16:15:00+07:00      Electronics         1     237.500      paid
   O1002        C002 2026-09-01 17:10:00+07:00             Home         2      91.000   shipped
   O1003        C003 2026-09-01 18:05:00+07:00            Books         3      35.073      paid
   O1004        C001 2026-09-02 15:45:00+07:00      Electronics         1      76.415      paid
   O1005        C004 2026-09-02 19:00:00+07:00          Fashion         2      62.400 cancelled

Pemeriksaan berhasil: seluruh baris berasal dari data valid.
```

JALANKAN QUERY ANALITIK PENJUALAN
Membuat file analyze_sales.py

scripts > analyze_sales.py > ...

```python

from pathlib import Path
import duckdb

PROJECT_DIR = Path(__file__).resolve().parent.parent
DB_PATH = PROJECT_DIR / "analytics.duckdb"
OUTPUT_PATH = PROJECT_DIR / "data" / "processed" / "daily_sales.csv"

query = """
SELECT
    CAST(order_ts AS DATE) AS order_date,
    product_category,
    SUM(quantity * unit_price * (1 - discount_pct)) AS net_sales
FROM raw.raw_orders
WHERE status IN ('paid', 'shipped')
GROUP BY 1, 2
ORDER BY 1, 2;
"""

def main():
    OUTPUT_PATH.parent.mkdir(parents=True, exist_ok=True)

    with duckdb.connect(str(DB_PATH), read_only=True) as con:
        result = con.execute(query)
        df = result.fetchdf()
        df.to_csv(OUTPUT_PATH, index=False)

        print("=== Daily Sales Analysis ===")
        print(df.to_string(index=False))
        print(f"\nJumlah baris hasil: {len(df)}")
        print(f"File tersimpan: {OUTPUT_PATH}")

if __name__ == "__main__":
    main()
```

Jalankan

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> python .\scripts\analyze_sales.py
=== Daily Sales Analysis ===
order_date product_category  net_sales
2026-09-01            Books     35.073
2026-09-01      Electronics    237.500
2026-09-01             Home     91.000
2026-09-02      Electronics     76.415
2026-09-02             Home    120.000
2026-09-03            Books     42.500
2026-09-03          Fashion     67.500
2026-09-04            Books     34.200
2026-09-04          Fashion     59.400
2026-09-04             Home     63.000
2026-09-05            Books     25.000
2026-09-05      Electronics    129.000
2026-09-05             Home    121.600
2026-09-06      Electronics    719.100
2026-09-06          Fashion     93.500
2026-09-07            Books     39.960
2026-09-07      Electronics     70.000
2026-09-07          Fashion     82.500
2026-09-07             Home    210.000
2026-09-08            Books     28.000
2026-09-08             Home     78.375
2026-09-09            Books     32.000
2026-09-09      Electronics     59.000
2026-09-09          Fashion    179.100
2026-09-09             Home    153.000
2026-09-10      Electronics    314.100
2026-09-10          Fashion     91.200
2026-09-10             Home    160.000

Jumlah baris hasil: 28
File tersimpan: D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\data\processed\daily_sales.csv
```

BUAT FILE dbt_project.yml

dbt_project > ! dbt_project.yml

```yaml
name: 'dbt_project'
version: '1.0.0'
config-version: 2

profile: 'uts_analytics'

model-paths: ["models"]
test-paths: ["tests"]

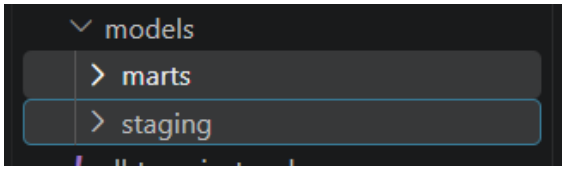
models:
  dbt_project:
    +materialized: table
    staging:
      +schema: staging
    marts:
      +schema: mart

```

BUAT FOLDER MODEL



BUAT FOLDER MART DAN STAGING



BUAT FILE staging/source.yml

dbt_project > models > staging > ! source.yml

```yaml
version: 2

sources:
  - name: raw
    schema: raw
    description: "Sumber data hasil validasi dan transformasi pada tahap sebelumnya."
    tables:
      - name: raw_orders
        description: "Data pesanan valid hasil transformasi yang disimpan di DuckDB."
        columns:
          - name: order_id
            description: "ID unik setiap pesanan."
          - name: customer_id
            description: "ID pelanggan yang melakukan pesanan."
          - name: order_ts
            description: "Timestamp pesanan."
          - name: quantity
            description: "Jumlah unit produk yang dipesan."
          - name: unit_price
            description: "Harga satuan produk."
          - name: discount_pct
            description: "Persentase diskon dalam rentang 0 sampai 1."
          - name: status
            description: "Status pesanan."
          - name: product_category
            description: "Kategori produk."
          - name: payment_method
            description: "Metode pembayaran."
          - name: gross_amount
            description: "Nilai pesanan sebelum diskon."
          - name: net_amount
            description: "Nilai pesanan setelah diskon."

```

BUAT FILE staging/stg_orders.sql

dbt_project > models > staging > stg_orders.sql

```sql

SELECT
    CAST(order_id AS VARCHAR) AS order_id,
    CAST(customer_id AS VARCHAR) AS customer_id,
    CAST(order_ts AS DATE) AS order_date,
    CAST(order_ts AS TIMESTAMP) AS order_timestamp,
    CAST(quantity AS BIGINT) AS quantity,
    CAST(unit_price AS DOUBLE) AS unit_price,
    CAST(discount_pct AS DOUBLE) AS discount_pct,
    CAST(status AS VARCHAR) AS status,
    CAST(product_category AS VARCHAR) AS product_category,
    CAST(payment_method AS VARCHAR) AS payment_method,
    CAST(gross_amount AS DOUBLE) AS gross_amount,
    CAST(net_amount AS DOUBLE) AS net_amount
FROM {{ source('raw', 'raw_orders') }}
```

BUAT FILE staging/stg_orders.yml

dbt_project > models > staging > ! stg_orders.yml

```yaml

version: 2

models:
  - name: stg_orders
    description: "Model staging untuk menstandarkan tipe data dan menyiapkan data pesanan valid."
    columns:
      - name: order_id
        description: "ID unik pesanan."
        data_tests:
          - unique
          - not_null

      - name: customer_id
        description: "ID pelanggan pemesan."
        data_tests:
          - not_null

      - name: order_date
        description: "Tanggal pesanan untuk analisis penjualan harian."

      - name: quantity
        description: "Jumlah unit yang dipesan."

      - name: net_amount
        description: "Nilai pesanan setelah diskon."

```

BUAT FILE marts/schma.yml

dbt_project > models > marts > ! schema.yml

```yaml

version: 2

models:
  - name: dim_customer
    description: "Dimensi pelanggan dengan satu baris untuk setiap customer."
    columns:
      - name: customer_id
        description: "Primary key dimensi pelanggan."
        data_tests:
          - unique
          - not_null

  - name: dim_product_category
    description: "Dimensi kategori produk dengan satu baris untuk setiap kategori."
    columns:
      - name: product_category
        description: "Nama kategori produk."
        data_tests:
          - unique
          - not_null

  - name: fct_orders
    description: "Tabel fakta transaksi dengan satu baris untuk setiap order."
    columns:
      - name: order_id
        description: "Primary key transaksi."
        data_tests:
          - unique
```

```yaml
      - name: customer_id
        description: "Foreign key yang mengacu ke dimensi pelanggan."
        data_tests:
          - not_null
          - relationships:
              arguments:
                to: ref('dim_customer')
                field: customer_id

      - name: product_category
        description: "Kategori produk pada transaksi."
        data_tests:
          - relationships:
              arguments:
                to: ref('dim_product_category')
                field: product_category

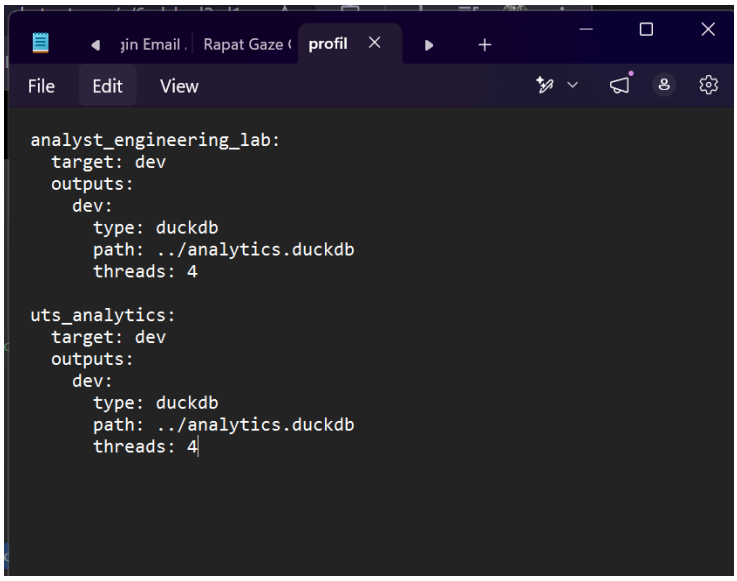
  - name: mart_daily_sales
    description: "Agregat penjualan berdasarkan tanggal, kategori produk, dan metode pembayaran."
    columns:
      - name: order_date
        description: "Tanggal penjualan."

      - name: product_category
        description: "Kategori produk."

      - name: payment_method
        description: "Metode pembayaran."

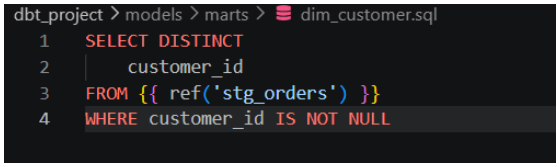
      - name: net_sales
        description: "Total penjualan setelah diskon untuk pesanan paid dan shipped."
```

Edit file profile.yml



BUAT MODEL DIMENSIONAL

dim.costumer



```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> python -c "import duckdb; con=duckdb.connect('analytics.duckdb'); con.execute(open('dbt_project/model/build_dim_customer.sql', encoding='utf-8').read()); print('dim_customer berhasil dibuat'); con.close()"
dim_customer berhasil dibuat
```

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> python -c "import duckdb; con=duckdb.connect('analytics.duckdb', read_only=True); print(con.execute('SELECT COUNT(*) AS total_rows, COUNT(DISTINCT customer_id) AS unique_customers FROM mart.dim_customer').fetchdf().to_string(index=False)); print(con.execute('SELECT * FROM mart.dim_customer ORDER BY customer_id LIMIT 10').fetchdf().to_string(index=False)); con.close()"
 total_rows  unique_customers
         25               25
customer_id
        C001
        C002
        C003
        C004
        C005
        C006
        C007
        C008
        C009
        C010
```

dim.product_category

dbt_project > models > marts > dim_product_category.sql

```sql

SELECT DISTINCT
    product_category
FROM {{ ref('stg_orders') }}
WHERE product_category IS NOT NULL
```

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> python -c "import duckdb; con=duckdb.connect('analytics.duckdb'); con.execute(open('dbt_project/model/dim_product_category.sql', encoding='utf-8').read()); print('dim_product_category berhasil dibuat'); con.close()"
dim_product_category berhasil dibuat
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> python -c "import duckdb; con=duckdb.connect('analytics.duckdb', read_only=True); print(con.execute('SELECT COUNT(*) AS total_rows, COUNT(DISTINCT product_category) AS unique_categories FROM mart.dim_product_category').fetchdf().to_string(index=False)); print(con.execute('SELECT * FROM mart.dim_product_category ORDER BY product_category').fetchdf().to_string(index=False)); con.close()"
 total_rows  unique_categories
          4                 4
product_category
           Books
     Electronics
         Fashion
            Home
```

Fct_orders

dbt_project > models > marts > fct_orders.sql

```sql
SELECT
    r.order_id,
    r.customer_id,
    r.product_category,
    r.order_timestamp AS order_ts,
    r.order_date,
    r.payment_method,
    r.status,
    r.quantity,
    r.unit_price,
    r.discount_pct,
    r.gross_amount,
    r.net_amount
FROM {{ ref('stg_orders') }} AS r
INNER JOIN {{ ref('dim_customer') }} AS c
    ON r.customer_id = c.customer_id
INNER JOIN {{ ref('dim_product_category') }} AS p
    ON r.product_category = p.product_category
```

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> python -c "import duckdb; con=duckdb.connect('analytics.duckdb'); con.execute(open('dbt_project/model/fct_orders.sql', encoding='utf-8').read()); print('fct_orders berhasil dibuat'); con.close()"
fct_orders berhasil dibuat
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS>
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> python -c "import duckdb; con=duckdb.connect('analytics.duckdb', read_only=True); print(con.execute('SELECT COUNT(*) AS total_rows, COUNT(DISTINCT order_id) AS unique_orders, COUNT(*) - COUNT(order_id) AS null_order_ids, COUNT(*) - COUNT(customer_id) AS null_customer_ids, COUNT(*) - COUNT(product_category) AS null_categories FROM mart.fct_orders').fetchdf().to_string(index=False)); print(con.execute('SELECT * FROM mart.fct_orders ORDER BY order_id LIMIT 5').fetchdf().to_string(index=False)); con.close()"
 total_rows  unique_orders  null_order_ids  null_customer_ids  null_categories
         30             30               0                  0                0
<string>:1: Pandas4Warning: The inplace keyword in DataFrame.drop is deprecated and will be removed in a future version (PDEP-8).
See https://pandas.pydata.org/docs/dev/whatsnew/v3.1.0.html#deprecation-inplace for more details.
order_id customer_id product_category                order_ts order_date payment_method    status  quantity  unit_price  discount_pct  gross_amount  net_amount
   O1001        C001      Electronics 2026-09-01 16:15:00+07:00 2026-09-01           card      paid         1      250.00          0.05        250.00     237.500
   O1002        C002             Home 2026-09-01 17:10:00+07:00 2026-09-01       transfer   shipped         2       45.50          0.00         91.00      91.000
   O1003        C003            Books 2026-09-01 18:05:00+07:00 2026-09-01        ewallet      paid         3       12.99          0.10         38.97      35.073
   O1004        C001      Electronics 2026-09-02 15:45:00+07:00 2026-09-02           card      paid         1       89.90          0.15         89.90      76.415
   O1005        C004          Fashion 2026-09-02 19:00:00+07:00 2026-09-02        ewallet cancelled         2       39.00          0.20         78.00      62.400
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS>
```

Mart_daily_sales

dbt_project > models > marts > mart_daily_sales.sql

```sql
SELECT
    order_date,
    product_category,
    payment_method,
    SUM(quantity * unit_price * (1 - discount_pct)) AS net_sales,
    SUM(quantity) AS total_quantity,
    COUNT(*) AS total_orders
FROM {{ ref('fct_orders') }}
WHERE status IN ('paid', 'shipped')
GROUP BY
    order_date,
    product_category,
    payment_method
```

```
01005        C004          Fashion 2026-09-02 19:00:00+07:00 20python -c "import duckdb; con=duckdb.connect('a
nalytics.duckdb'); con.execute(open('dbt_project/model/mart_daily_sales.sql', encoding='utf-8').read()); print('m
art_daily_sales berhasil dibuat'); con.close()"gas\praktikum\UTS>
mart_daily_sales berhasil dibuat
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS> python -c "import duckdb; con=duckdb.connect('a
nalytics.duckdb', read_only=True); print(con.execute('SELECT COUNT(*) AS total_groups, SUM(total_orders) AS total
_paid_shipped_orders, SUM(net_sales) AS total_net_sales FROM mart.mart_daily_sales').fetchdf().to_string(index=Fa
lse)); print(con.execute('SELECT * FROM mart.mart_daily_sales ORDER BY order_date, product_category, payment_meth
od LIMIT 10').fetchdf().to_string(index=False)); con.close()"
 total_groups  total_paid_shipped_orders  total_net_sales
           28                      28.0         3412.023
order_date product_category payment_method  net_sales  total_quantity  total_orders
2026-09-01           Books        ewallet     35.073             3.0             1
2026-09-01      Electronics           card    237.500             1.0             1
2026-09-01            Home       transfer     91.000             2.0             1
2026-09-02      Electronics           card     76.415             1.0             1
2026-09-02            Home       transfer    120.000             1.0             1
2026-09-03           Books           card     42.500             5.0             1
2026-09-03         Fashion        ewallet     67.500             1.0             1
2026-09-04           Books           card     34.200             2.0             1
2026-09-04         Fashion        ewallet     59.400             3.0             1
2026-09-04            Home       transfer     63.000             4.0             1
```

MENJALANKAN dbt debug

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\dbt_project> dbt debug
02:59:45  Running with dbt=1.12.6
02:59:46  dbt version: 1.12.6
02:59:46  python version: 3.11.9
02:59:46  python path: D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\.venv\Scripts\python.exe
02:59:46  os info: Windows-10-10.0.26300-SP0
02:59:46  Using profiles dir at C:\Users\ASUS\.dbt
02:59:46  Using profiles.yml file at C:\Users\ASUS\.dbt\profiles.yml
02:59:46  Using dbt_project.yml file at D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\dbt_project\dbt_pro
ject.yml
02:59:46  adapter type: duckdb
02:59:46  adapter version: 1.11.0
02:59:46  Configuration:
02:59:46    profiles.yml file [OK found and valid]
02:59:46    dbt_project.yml file [OK found and valid]
02:59:46  Required dependencies:
02:59:46   - git [OK found]

02:59:46  Connection:
02:59:46    database: analytics
02:59:46    schema: main
02:59:46    path: ../analytics.duckdb
02:59:46    config_options: None
02:59:46    extensions: None
02:59:46    settings: {}
02:59:46    external_root: .
02:59:46    use_credential_provider: None
02:59:46    attach: None
02:59:46    filesystems: None
02:59:46    remote: None
02:59:46    plugins: None
02:59:46    disable_transactions: False
02:59:46  Registered adapter: duckdb=1.11.0
02:59:54    Connection test: [OK connection ok]

02:59:54  All checks passed!
```

JALANKAN dbt parse UNTUK MELIHAT APAKAH KONFIGURASI DAN FILE MODEL SUDAH TERBACA DENGAN BENAR

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\dbt_project> dbt parse
03:03:13  Running with dbt=1.12.6
03:03:14  Registered adapter: duckdb=1.11.0
03:03:15  Unable to do partial parsing because saved manifest not found. Starting full parse.
03:03:21  Performance info: D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\dbt_project\target\perf_info.js
```

JALANKAN dbt ls UNTUK MELIHAT MODEL YANG DIKENALI DBT

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\dbt_project> dbt ls
03:03:42  Running with dbt=1.12.6
03:03:42  Registered adapter: duckdb=1.11.0
03:03:44  Found 5 models, 12 data tests, 1 source, 500 macros
dbt_project.marts.dim_customer
dbt_project.marts.dim_product_category
dbt_project.marts.fct_orders
dbt_project.marts.mart_daily_sales
dbt_project.staging.stg_orders
source:dbt_project.raw.raw_orders
dbt_project.marts.not_null_dim_customer_customer_id
dbt_project.marts.not_null_dim_product_category_product_category
dbt_project.marts.not_null_fct_orders_customer_id
dbt_project.marts.not_null_fct_orders_order_id
dbt_project.staging.not_null_stg_orders_customer_id
dbt_project.staging.not_null_stg_orders_order_id
dbt_project.marts.relationships_fct_orders_customer_id__customer_id__ref_dim_customer_
dbt_project.marts.relationships_fct_orders_product_category__product_category__ref_dim_product_category_
dbt_project.marts.unique_dim_customer_customer_id
dbt_project.marts.unique_dim_product_category_product_category
dbt_project.marts.unique_fct_orders_order_id
dbt_project.staging.unique_stg_orders_order_id
```

JALANKAN dbt run untuk menjalankan ke lima model

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\dbt_project> dbt run
03:21:27  Running with dbt=1.12.6
03:21:28  Registered adapter: duckdb=1.11.0
03:21:31  Found 5 models, 12 data tests, 1 source, 500 macros
03:21:31
03:21:31  Concurrency: 4 threads (target='dev')
03:21:31
03:21:32  1 of 5 START sql table model main_staging.stg_orders ........................... [RUN]
03:21:32  1 of 5 OK created sql table model main_staging.stg_orders ...................... [OK in 0.20s]
03:21:32  2 of 5 START sql table model main_mart.dim_customer ............................ [RUN]
03:21:32  3 of 5 START sql table model main_mart.dim_product_category .................... [RUN]
03:21:32  2 of 5 OK created sql table model main_mart.dim_customer ....................... [OK in 0.29s]
03:21:32  3 of 5 OK created sql table model main_mart.dim_product_category ............... [OK in 0.32s]
03:21:32  4 of 5 START sql table model main_mart.fct_orders .............................. [RUN]
03:21:33  4 of 5 OK created sql table model main_mart.fct_orders ......................... [OK in 0.16s]
03:21:33  5 of 5 START sql table model main_mart.mart_daily_sales ........................ [RUN]
03:21:33  5 of 5 OK created sql table model main_mart.mart_daily_sales ................... [OK in 0.11s]
03:21:33
03:21:33  Finished running 5 table models in 0 hours 0 minutes and 1.70 seconds (1.70s).
03:21:33
03:21:33  Completed successfully
03:21:33
03:21:33  Done. PASS=5 WARN=0 ERROR=0 SKIP=0 NO-OP=0 REUSED=0 TOTAL=5
```

JALANKAN dbt test

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\dbt_project> dbt test
03:29:06  Running with dbt=1.12.6
03:29:06  Registered adapter: duckdb=1.11.0
03:29:08  Found 5 models, 12 data tests, 1 source, 500 macros
03:29:08
03:29:08  Concurrency: 4 threads (target='dev')
03:29:08
03:29:08  1 of 12 START test not_null_dim_customer_customer_id ........................... [RUN]
03:29:08  2 of 12 START test not_null_dim_product_category_product_category .............. [RUN]
03:29:08  3 of 12 START test not_null_fct_orders_customer_id ............................. [RUN]
03:29:08  4 of 12 START test not_null_fct_orders_order_id ................................ [RUN]
03:29:09  1 of 12 PASS not_null_dim_customer_customer_id ................................. [PASS in 0.25s]
03:29:09  4 of 12 PASS not_null_fct_orders_order_id ...................................... [PASS in 0.25s]
03:29:09  2 of 12 PASS not_null_dim_product_category_product_category .................... [PASS in 0.25s]
03:29:09  3 of 12 PASS not_null_fct_orders_customer_id ................................... [PASS in 0.25s]
03:29:09  5 of 12 START test not_null_stg_orders_customer_id ............................. [RUN]
03:29:09  6 of 12 START test not_null_stg_orders_order_id ................................ [RUN]
03:29:09  7 of 12 START test relationships_fct_orders_customer_id__customer_id__ref_dim_customer_  [RUN]
03:29:09  8 of 12 START test relationships_fct_orders_product_category__product_category__ref_dim_product_categor
y_  [RUN]
03:29:09  5 of 12 PASS not_null_stg_orders_customer_id ................................... [PASS in 0.21s]
03:29:09  6 of 12 PASS not_null_stg_orders_order_id ...................................... [PASS in 0.21s]
03:29:09  9 of 12 START test unique_dim_customer_customer_id ............................. [RUN]
03:29:09  10 of 12 START test unique_dim_product_category_product_category ............... [RUN]
03:29:09  8 of 12 PASS relationships_fct_orders_product_category__product_category__ref_dim_product_category_  [P
ASS in 0.35s]
03:29:09  7 of 12 PASS relationships_fct_orders_customer_id__customer_id__ref_dim_customer_  [PASS in 0.42s]
03:29:09  9 of 12 PASS unique_dim_customer_customer_id ................................... [PASS in 0.22s]
03:29:09  11 of 12 START test unique_fct_orders_order_id ................................. [RUN]
03:29:09  10 of 12 PASS unique_dim_product_category_product_category ..................... [PASS in 0.23s]
03:29:09  12 of 12 START test unique_stg_orders_order_id ................................. [RUN]
03:29:09  11 of 12 PASS unique_fct_orders_order_id ....................................... [PASS in 0.18s]
03:29:10  12 of 12 PASS unique_stg_orders_order_id ....................................... [PASS in 0.17s]
03:29:10
03:29:10  Finished running 12 data tests in 0 hours 0 minutes and 1.82 seconds (1.82s).
03:29:10
03:29:10  Completed successfully
03:29:10
03:29:10  Done. PASS=12 WARN=0 ERROR=0 SKIP=0 NO-OP=0 REUSED=0 TOTAL=12
```

VALIDASI JUMLAH BARIS TIAP TABEL YANG DIHASILKAN OLEH MODEL

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\dbt_project> python -c "import duckdb; c=duckdb.
connect('../analytics.duckdb', read_only=True); print(c.execute('''SELECT table_schema, table_name FROM informati
on_schema.tables WHERE table_name IN ('stg_orders','dim_customer','dim_product_category','fct_orders','mart_daily
_sales') ORDER BY table_schema, table_name''').fetchall()); print(c.execute('''SELECT 'stg_orders' AS model, COUN
T(*) AS jumlah_baris FROM main_staging.stg_orders UNION ALL SELECT 'dim_customer', COUNT(*) FROM main_mart.dim_cu
stomer UNION ALL SELECT 'dim_product_category', COUNT(*) FROM main_mart.dim_product_category UNION ALL SELECT 'fc
t_orders', COUNT(*) FROM main_mart.fct_orders UNION ALL SELECT 'mart_daily_sales', COUNT(*) FROM main_mart.mart_d
aily_sales'''').fetchall()); c.close()"
[('main_mart', 'dim_customer'), ('main_mart', 'dim_product_category'), ('main_mart', 'fct_orders'), ('main_mart',
 'mart_daily_sales'), ('main_staging', 'stg_orders'), ('mart', 'dim_customer'), ('mart', 'dim_product_category'),
 ('mart', 'fct_orders'), ('mart', 'mart_daily_sales')]
[('stg_orders', 30), ('dim_customer', 25), ('dim_product_category', 4), ('fct_orders', 30), ('mart_daily_sales',
28)]
```

Dim_customer = 25 baris
Dim_product_category = 4 baris
Fact_orders = 30 baris
Mart_dailt_sales = 28 baris
Stg_orders = 30 baris

JALANKAN DOKUMENTASI DENGAN MENGGUNAKAN dbt docs generative

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\dbt_project> dbt docs generate
03:50:04  Running with dbt=1.12.6
03:50:05  Registered adapter: duckdb=1.11.0
03:50:06  Found 5 models, 12 data tests, 1 source, 500 macros
03:50:06
03:50:06  Concurrency: 4 threads (target='dev')
03:50:06
03:50:08  Building catalog
03:50:09  Catalog written to D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\dbt_project\target\catalog.jso
n
```

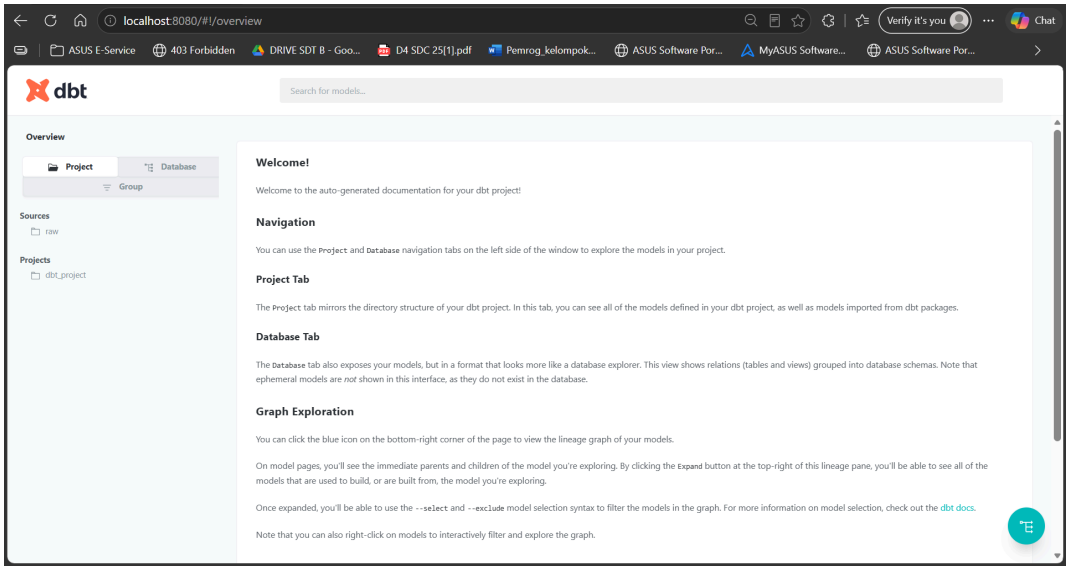
BUKA SERVER DUCKDB DENGAN MENJALANKAN dbt docs server

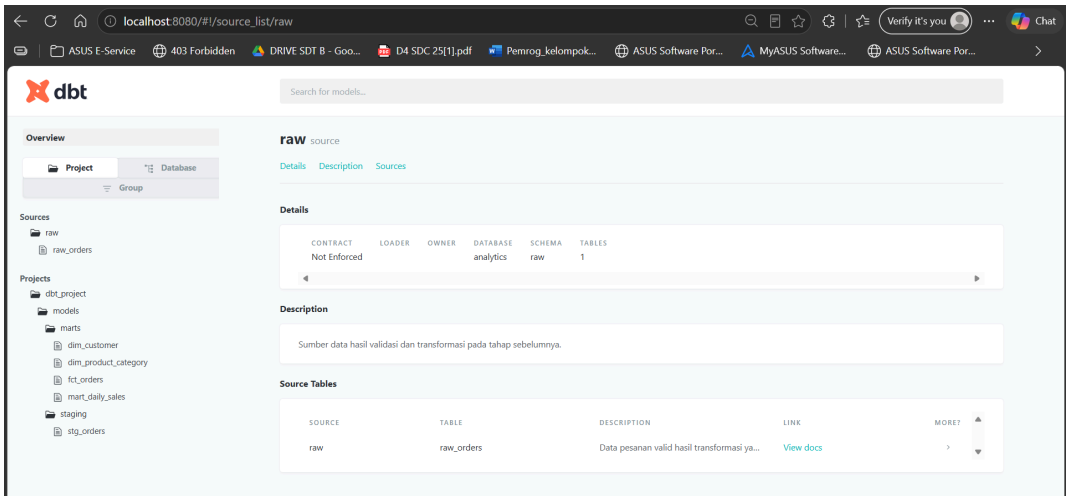
```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\dbt_project> dbt docs serve
03:53:03  Running with dbt=1.12.6
Serving docs at 8080
To access from your browser, navigate to: http://localhost:8080

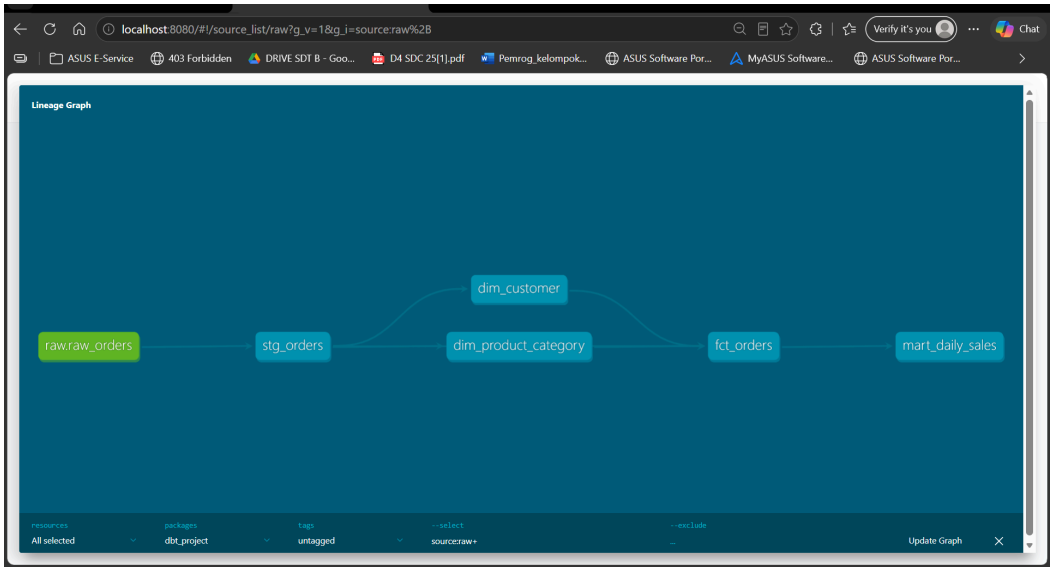


Press Ctrl+C to exit.
```

TAMPILAN AWAL


localhost:8080/#!/overview
dbt
Search for models...
Overview
Project
Database
Group
Sources
raw
Projects
dbt_project
Welcome!
Welcome to the auto-generated documentation for your dbt project!
Navigation
You can use the Project and Database navigation tabs on the left side of the window to explore the models in your project.
Project Tab
The Project tab mirrors the directory structure of your dbt project. In this tab, you can see all of the models defined in your dbt project, as well as models imported from dbt packages.
Database Tab
The Database tab also exposes your models, but in a format that looks more like a database explorer. This view shows relations (tables and views) grouped into database schemas. Note that ephemeral models are not shown in this interface, as they do not exist in the database.
Graph Exploration
You can click the blue icon on the bottom-right corner of the page to view the lineage graph of your models.
On model pages, you'll see the immediate parents and children of the model you're exploring. By clicking the Expand button at the top-right of this lineage pane, you'll be able to see all of the models that are used to build, or are built from, the model you're exploring.
Once expanded, you'll be able to use the --select and --exclude model selection syntax to filter the models in the graph. For more information on model selection, check out the dbt docs.
Note that you can also right-click on models to interactively filter and explore the graph.


localhost:8080/#!/source_list/raw
dbt
Search for models...
Overview
Project
Database
Group
Sources
raw
raw_orders
Projects
dbt_project
models
marts
dim_customer
dim_product_category
fct_orders
mart_daily_sales
staging
stg_orders
raw source
Details
Description
Sources
Details
CONTRACT
LOADER
OWNER
DATABASE
SCHEMA
TABLES
Not Enforced
analytics
raw
1
Description
Sumber data hasil validasi dan transformasi pada tahap sebelumnya.
Source Tables
SOURCE
TABLE
DESCRIPTION
LINK
MORE?
raw
raw_orders
Data pesanan valid hasil transformasi ya...
View docs


localhost:8080/#!/source_list/raw?g_v=1&g_i=source:raw%2B
Lineage Graph
dim_customer
raw.raw_orders
stg_orders
dim_product_category
fct_orders
mart_daily_sales
resources
All selected
packages
dbt_project
tags
untagged
--select
source:raw+
--exclude
Update Graph

G. MARKDOWN PRAKTIKUM (README.md)

H. PERTANYAAN ANALITIK DAN CARA JAWAB

BUAT FILE PYTHON UNTUK MENJALANKAN QUERY JAWABAN

dbt_project > analisis_penjualan.py > ...

```python

import duckdb
import json
from pathlib import Path
from collections import Counter

# ============================================================
# KONFIGURASI
# ============================================================

DB_PATH = r"D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\analytics.duckdb"
REJECTED_PATH = Path(r"D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\data\rejected\rejected_orders.jsor

# Koneksi ke database DuckDB
con = duckdb.connect(str(DB_PATH), read_only=True)


def tampilkan_hasil(judul, query):
    """Menjalankan query SQL dan menampilkan hasilnya."""
    print("\n" + "=" * 70)
    print(judul)
    print("=" * 70)

    hasil = con.execute(query).df()
    print(hasil.to_string(index=False))
    return hasil


```

```python
# ============================================================
# 1. BERAPA NILAI PENJUALAN BERSIH PER HARI?
# ============================================================

sql_penjualan_harian = """
SELECT
    CAST(order_ts AS DATE) AS tanggal,
    ROUND(SUM(net_amount), 2) AS penjualan_bersih
FROM raw.raw_orders
GROUP BY CAST(order_ts AS DATE)
ORDER BY tanggal;
"""

tampilkan_hasil(
    "1. PENJUALAN BERSIH PER HARI",
    sql_penjualan_harian
)


# ============================================================
# 2. KATEGORI PRODUK MANA YANG PENJUALAN BERSIHNYA TERTINGGI?
# ============================================================


sql_penjualan_kategori = """
SELECT
    product_category,
    ROUND(SUM(net_amount), 2) AS penjualan_bersih
FROM raw.raw_orders
GROUP BY product_category
ORDER BY penjualan_bersih DESC;
"""


hasil_kategori = tampilkan_hasil(
    "2. PENJUALAN BERSIH MENURUT KATEGORI PRODUK",
    sql_penjualan_kategori
)
```

```python
if not hasil_kategori.empty:
    tertinggi = hasil_kategori.iloc[0]
    print(
        f"\nKategori dengan penjualan bersih tertinggi: "
        f"{tertinggi['product_category']} "
        f"(nilai: {tertinggi['penjualan_bersih']})"
    )


# ============================================================
# 3. BAGAIMANA JUMLAH DAN NILAI PESANAN MENURUT STATUS?
# ============================================================

sql_status_pesanan = """
SELECT
    status,
    COUNT(DISTINCT order_id) AS jumlah_pesanan,
    ROUND(
        SUM(quantity * unit_price * (1 - COALESCE(discount_pct, 0))),
        2
    ) AS nilai_penjualan_bersih
FROM raw.raw_orders
GROUP BY status
ORDER BY jumlah_pesanan DESC;
"""

tampilkan_hasil(
    "3. JUMLAH DAN NILAI PESANAN MENURUT STATUS",
    sql_status_pesanan
)
```

```python
# ============================================================
# 4. METODE PEMBAYARAN MANA YANG PALING SERING DIGUNAKAN?
# ============================================================

sql_metode_pembayaran = """
SELECT
    payment_method,
    COUNT(DISTINCT order_id) AS jumlah_pesanan
FROM raw.raw_orders
GROUP BY payment_method
ORDER BY jumlah_pesanan DESC;
"""

hasil_pembayaran = tampilkan_hasil(
    "4. FREKUENSI METODE PEMBAYARAN",
    sql_metode_pembayaran
)

if not hasil_pembayaran.empty:
    terpopuler = hasil_pembayaran.iloc[0]
    print(
        f"\nMetode pembayaran paling sering digunakan: "
        f"{terpopuler['payment_method']} "
        f"({terpopuler['jumlah_pesanan']} pesanan)"
    )
```

```python
# ============================================================
# 5. BERAPA RECORD DITOLAK DAN APA PENYEBAB TERBESARNYA?
# ============================================================

print("\n" + "=" * 70)
print("5. RECORD YANG DITOLAK OLEH DATA CONTRACT")
print("=" * 70)

if REJECTED_PATH.exists():
    records = []

    with open(REJECTED_PATH, "r", encoding="utf-8") as file:
        for baris in file:
            baris = baris.strip()
            if baris:
                try:
                    records.append(json.loads(baris))
                except json.JSONDecodeError:
                    print("Peringatan: ditemukan baris JSON yang tidak valid.")

    print(f"Jumlah record ditolak: {len(records)}")

    penyebab = Counter()

    for record in records:
        # Mendukung beberapa kemungkinan nama field alasan.
        alasan = (
            record.get("reason")
            or record.get("rejection_reason")
            or record.get("error")
            or record.get("errors")
            or record.get("reason_code")
            or record.get("validation_errors")
        )
```

```
161
162             if alasan is None:
163                 alasan = "Alasan tidak tercantum"
164
165             if isinstance(alasan, list):
166                 for item in alasan:
167                     penyebab[str(item)] += 1
168             elif isinstance(alasan, dict):
169                 for key, value in alasan.items():
170                     penyebab[f"{key}: {value}"] += 1
171             else:
172                 penyebab[str(alasan)] += 1
173
174         print("\nFrekuensi penyebab penolakan:")
175
176         for alasan, jumlah in penyebab.most_common():
177             print(f"- {alasan}: {jumlah}")
178
179         if penyebab:
180             alasan_terbesar, jumlah_terbesar = penyebab.most_common(1)[0]
181             print(
182                 f"\nPenyebab penolakan terbanyak: {alasan_terbesar} "
183                 f"({jumlah_terbesar} kemunculan)"
184             )
185
186     else:
187         print(f"File rejected tidak ditemukan: {REJECTED_PATH}")
188         print("Periksa kembali lokasi file rejected_orders.jsonl.")
189
190     con.close()
191
```

JALANKAN FILE PYTHON

```
(.venv) PS D:\Sem 5\Data Processing Workflow\tugas\praktikum\UTS\dbt_project> python analisis_penjualan.py

======================================================================
1. PENJUALAN BERSIH PER HARI
======================================================================
   tanggal  penjualan_bersih
2026-09-01            363.57
2026-09-02            258.82
2026-09-03            584.05
2026-09-04            156.60
2026-09-05            275.60
2026-09-06            812.60
2026-09-07            402.46
2026-09-08            106.38
2026-09-09            423.10
2026-09-10            565.30

======================================================================
2. PENJUALAN BERSIH MENURUT KATEGORI PRODUK
======================================================================
product_category  penjualan_bersih
     Electronics           2079.17
            Home            996.98
         Fashion            635.60
           Books            236.73

Kategori dengan penjualan bersih tertinggi: Electronics (nilai: 2079.17)
```

```
================================================================
3. JUMLAH DAN NILAI PESANAN MENURUT STATUS
================================================================
   status  jumlah_pesanan  nilai_penjualan_bersih
     paid              20                 2443.85
  shipped               8                  968.18
cancelled               1                   62.40
 refunded               1                  474.05

================================================================
4. FREKUENSI METODE PEMBAYARAN
================================================================
payment_method  jumlah_pesanan
          card              14
       ewallet               8
      transfer               8

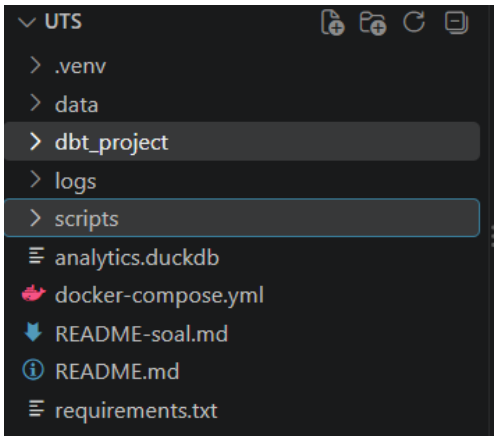
Metode pembayaran paling sering digunakan: card (14 pesanan)

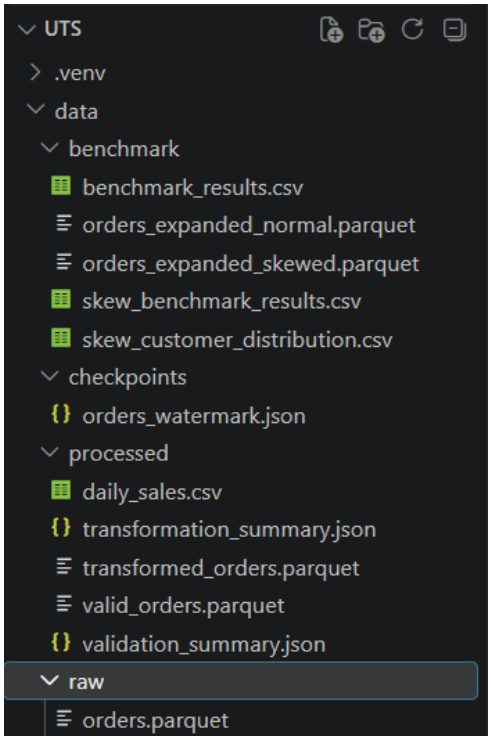
================================================================
5. RECORD YANG DITOLAK OLEH DATA CONTRACT
================================================================
Jumlah record ditolak: 6

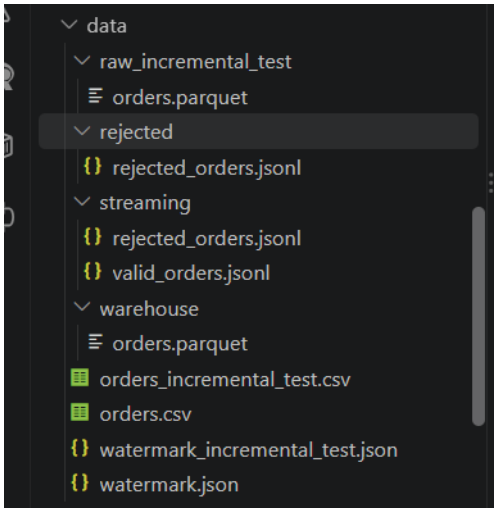
Frekuensi penyebab penolakan:
- {'field': 'quantity', 'message': 'Input should be greater than 0', 'type': 'greater_than'}: 1
- {'field': 'unit_price', 'message': 'Input should be greater than or equal to 0', 'type': 'greater_than_equal'}:
 1
- {'field': 'discount_pct', 'message': 'Input should be less than or equal to 1', 'type': 'less_than_equal'}: 1
- {'field': 'status', 'message': "Input should be 'paid', 'shipped', 'cancelled' or 'refunded'", 'type': 'literal
_error'}: 1
- {'field': 'customer_id', 'message': 'Input should be a valid string', 'type': 'string_type'}: 1
- {'field': 'order_ts', 'message': 'Input should be a valid datetime', 'type': 'datetime_type'}: 1

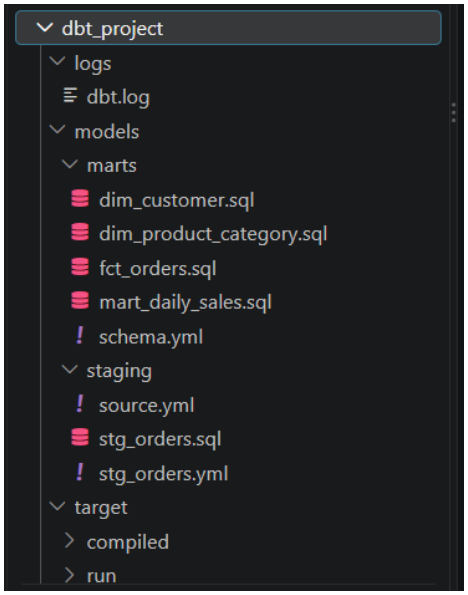
Penyebab penolakan terbanyak: {'field': 'quantity', 'message': 'Input should be greater than 0', 'type': 'greater
 than'} (1 kemunculan)
```

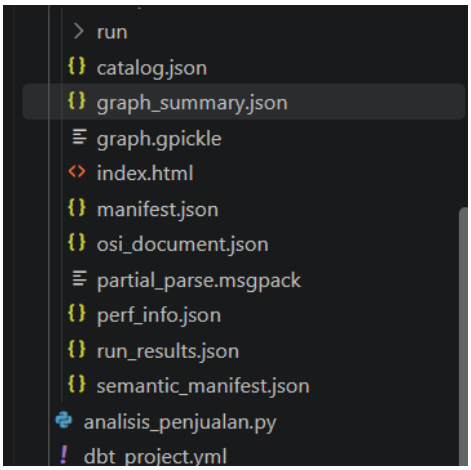
I. STRUKTUR FOLDER HASIL AKHIR YANG DISARANKAN











Berikut adalah link terusan dari laporan ini :
https://docs.google.com/document/d/1_tCACwNT5PuYdezt1o0pai5WBF1xD_NKQZ6wECU2eQ0/edit?usp=sharing